

STUDENT PERFORMANCE & PLACEMENT PREDICTOR

AI & MACHINE LEARNING MINI PROJECT

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

Submitted by

Student Name : [Student Name]

Roll Number : [Roll Number]

PRN : [PRN]

Division : [Division]

Under the guidance of

[Guide / Professor Name]

[Department of Computer Science & Engineering]

[College / University Name]

[City, State]

Academic Year [20XX–20XX]

[College / University Name]
[Department of Computer Science & Engineering]

CERTIFICATE

This is to certify that the mini project report entitled “**Student Performance & Placement Predictor**” submitted by **[Student Name]** (Roll No. [Roll Number], PRN [PRN], Division [Division]) is a bonafide record of the work carried out under my supervision and guidance, in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Science & Engineering** of [College / University Name] during the academic year [20XX–20XX].

To the best of my knowledge, the work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma.

[Guide / Professor Name]
Project Guide
Department of CSE

[Head of Department]
Head of Department
Department of CSE

External Examiner
Date: _____

[Principal / Director]
Place: _____

DECLARATION

I, **[Student Name]**, a student of B.Tech Computer Science & Engineering (Roll No. [Roll Number], PRN [PRN]), hereby declare that the mini project report entitled “**Student Performance & Placement Predictor**” submitted to [College / University Name] is a record of my own academic work carried out under the guidance of [Guide / Professor Name].

I further declare that the dataset used in this project is synthetic and was generated as part of the project, that all results reported here were produced by the project's own code, and that the work of other authors has been duly acknowledged and cited. This report has not been submitted, in part or in full, for any other degree or diploma.

Place: _____
Date: _____

[Student Name]
[Roll Number]

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, **[Guide / Professor Name]**, for their valuable guidance, encouragement and constructive feedback throughout this mini project. Their suggestions on methodology and on the honest reporting of results shaped this work.

I am thankful to **[Head of Department]**, Head of the Department of Computer Science & Engineering, for providing the facilities and an environment that made this project possible, and to all faculty members of the department for the knowledge of machine learning, data science and software engineering on which this project builds.

I also thank the management and Principal of **[College / University Name]** for their support, and the laboratory staff for their assistance.

Finally, I am grateful to my family and friends for their constant encouragement and patience, and to the developers of the open-source tools — Python, scikit-learn, XGBoost, SHAP, pandas, Matplotlib, Seaborn, Plotly and Streamlit — on which this project relies.

[Student Name]

ABSTRACT

Identifying students who are not yet prepared for campus recruitment usually happens late and informally, because academic, skill and experience indicators are scattered and are rarely assessed together. This project develops the Student Performance & Placement Predictor, a machine-learning system that combines these indicators into early, explainable estimates.

The system predicts three outputs from 14 inputs — CGPA, attendance, backlogs, coding, aptitude, communication, technical-skills and extracurricular scores, projects, certifications, internship, hackathon participation, programming languages and study hours: an academic-performance category (Poor to Excellent), a placement-readiness category (Not Ready to Highly Ready) and an estimated placement probability. Because consented real student records were unavailable, a synthetic dataset of 1,200 students was generated with deliberately injected missing values, duplicates and invalid entries. A leakage-free scikit-learn pipeline performs cleaning, median imputation, scaling, one-hot encoding and variance-based feature selection. Logistic Regression, Decision Tree, Random Forest, Gradient Boosting and XGBoost were compared for each task using 5-fold cross-validation on 960 training records, and the selected models were evaluated once on 240 held-out records.

XGBoost was selected for academic performance (test accuracy 80.0 %, macro-F1 0.812), Logistic Regression for placement readiness (accuracy 86.7 %, macro-F1 0.835) and XGBoost for placement probability (mean absolute error 7.07 percentage points, R^2 0.930). Coding score, internship experience, CGPA and projects were the most influential placement factors. SHAP values explain every individual prediction. A Streamlit application provides a dashboard, a validated prediction form, interactive analytics and project documentation; all 18 test cases and 3 application tests passed.

The main limitation is the synthetic data: the results show that the pipeline recovers the simulated relationships, not that it predicts real placements, and no prediction is a guarantee. Future work includes validation on real institutional data, résumé analysis, personalised recommendations and cloud deployment.

Keywords: educational data mining, placement prediction, student performance, XGBoost, logistic regression, SHAP, explainable AI, Streamlit

TABLE OF CONTENTS

Abstract.....	iv
List of Figures.....	viii
List of Tables.....	ix
1 Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Motivation.....	1
1.4 Objectives.....	2
1.5 Scope.....	2
1.6 Proposed Solution.....	2
1.7 Expected Outcome.....	2
2 Literature Review.....	3
2.1 Student Performance Prediction.....	3
2.2 Educational Data Mining.....	3
2.3 Machine Learning in Education.....	3
2.4 Placement Prediction.....	3
2.5 Classification Algorithms.....	4
2.6 Explainable AI.....	4
2.7 Summary of Reviewed Literature.....	4
3 System Analysis.....	6
3.1 Existing System.....	6
3.2 Proposed System.....	6
3.3 Advantages.....	6
3.4 Limitations.....	6
3.5 Functional Requirements.....	7
3.6 Non-functional Requirements.....	7
4 Technology Used.....	8
5 Dataset.....	9
5.1 Nature of the Dataset.....	9
5.2 Dataset Size and Structure.....	9
5.3 Feature Description.....	10
5.4 Missing Values.....	10
5.5 Class Distribution.....	11
5.6 Descriptive Statistics.....	11
5.7 Representative Records.....	12
6 Data Preprocessing.....	13
6.1 Data Cleaning.....	13
6.2 Missing-value Handling.....	13
6.3 Duplicate Handling.....	14
6.4 Outlier Handling.....	14
6.5 Encoding.....	15
6.6 Feature Scaling.....	15
6.7 Feature Engineering and Selection.....	15
6.8 Train/Test Split and Prevention of Data Leakage.....	15
7 Exploratory Data Analysis.....	17
7.1 Dataset Distribution.....	17
7.2 Univariate Distributions.....	17

7.3 Correlation Analysis.....	19
7.4 Relationships with Placement Readiness.....	19
7.5 Feature Importance.....	20
8 Machine Learning Methodology.....	21
8.1 Logistic Regression.....	21
8.2 Decision Tree.....	21
8.3 Random Forest.....	22
8.4 Gradient Boosting.....	22
8.5 XGBoost.....	22
8.6 Training, Validation and Prediction.....	23
9 Model Evaluation.....	24
9.1 Academic Performance Classification.....	24
9.2 Placement Readiness Classification.....	24
9.3 Placement Probability Regression.....	24
9.4 Cross-validation Results and Model Selection.....	25
9.5 Confusion Matrices.....	26
9.6 Per-class Results.....	26
9.7 ROC Curves.....	27
9.8 Regression Diagnostics.....	27
9.9 Fairness Audit.....	27
9.10 Interpretation.....	28
10 Explainable AI.....	29
10.1 Why Interpretability Matters in Education.....	29
10.2 Global Feature Importance.....	29
10.3 SHAP Explanations.....	30
10.4 Example Prediction Explanation.....	30
11 System Design.....	32
11.1 System Architecture Diagram.....	32
11.2 System Architecture (Component View).....	33
11.3 Data-flow Diagram.....	33
11.4 ML Pipeline Diagram.....	33
11.5 Application Workflow.....	34
12 Implementation.....	35
12.1 Project Structure.....	35
12.2 Dataset Loading.....	35
12.3 Preprocessing.....	35
12.4 Model Training.....	35
12.5 Model Saving.....	36
12.6 Prediction.....	36
12.7 Streamlit Interface.....	36
12.8 Visualisation.....	36
12.9 Explainability.....	36
13 Application Screenshots.....	37
14 Testing.....	41
14.1 Test Approach.....	41
14.2 Test Cases.....	41
14.3 Application Tests.....	42
15 Results.....	44
15.1 Selected Models and Metrics.....	44
15.2 Prediction Examples.....	44

15.3 Application Performance.....	44
15.4 Key Observations.....	44
16 Limitations.....	45
16.1 Dataset Limitations.....	45
16.2 Synthetic-data Limitations.....	45
16.3 Dataset Size.....	45
16.4 Potential Bias.....	45
16.5 Model Generalisation.....	45
16.6 Placement Outcomes Depend on Many Factors.....	45
16.7 Predictions Are Not Guarantees.....	45
16.8 Need for Real-world Validation.....	45
17 Future Scope.....	46
18 Conclusion.....	47
References.....	48
Appendix A — Dataset Sample.....	50
Appendix B — Important Code.....	51
Appendix C — Model Metrics.....	53
Appendix D — Additional Screenshots.....	55

LIST OF FIGURES

Figure 6.1: End-to-end ML pipeline.....	13
Figure 6.2: Missing values per column in the raw data.....	14
Figure 7.1: Distribution of the three target variables.....	17
Figure 7.2: CGPA distribution.....	17
Figure 7.3: Attendance distribution.....	17
Figure 7.4: Coding / DSA score distribution.....	17
Figure 7.5: Backlog distribution.....	17
Figure 7.6: Project-count distribution.....	18
Figure 7.7: Certification distribution.....	18
Figure 7.8: Internship distribution.....	18
Figure 7.9: Correlation heatmap of numeric features and placement probability.....	19
Figure 7.10: CGPA vs placement readiness.....	19
Figure 7.11: Coding score vs placement readiness.....	19
Figure 7.12: Projects vs placement readiness.....	20
Figure 7.13: Attendance vs CGPA by academic performance.....	20
Figure 7.14: Internship and hackathon participation vs placement readiness.....	20
Figure 7.15: Permutation feature importance of the three selected models.....	20
Figure 9.1: Model comparison by cross-validation score (selected model in teal).....	25
Figure 9.2: Academic performance (XGBoost).....	26
Figure 9.3: Placement readiness (Logistic Regression).....	26
Figure 9.4: One-vs-rest ROC curves of the selected classifiers (test set).....	27
Figure 9.5: Predicted vs actual placement probability and residual distribution (XGBoost, test set).....	27
Figure 10.1: SHAP summary: placement probability model.....	30
Figure 10.2: SHAP summary: readiness model ('Highly Ready' class).....	30
Figure 10.3: Explanation panel in the application for the same student.....	31
Figure 11.1: System architecture: request flow from user to results.....	32
Figure 11.2: Component architecture: presentation layer, application logic and stored artefacts.....	33
Figure 11.3: Level-1 data-flow diagram of the prediction process.....	33
Figure 11.4: Application workflow.....	34
Figure 13.1: Dashboard.....	37
Figure 13.2: Student input form.....	38
Figure 13.3: Prediction screen for the average-student sample.....	39
Figure 13.4: Academic performance result.....	39
Figure 13.5: Placement readiness result.....	39
Figure 13.6: Estimated placement probability.....	39
Figure 13.7: Feature importance in the Analytics page.....	40
Figure 13.8: Analytics: dataset relationships.....	40
Figure 13.9: Analytics: model evaluation tab.....	40
Figure 13.10: About section.....	40
Figure 14.1: Validation in the application.....	43
Figure C.1: Confusion matrices of all five academic-performance models (test set).....	54
Figure C.2: Confusion matrices of all five placement-readiness models (test set).....	54
Figure D.1: Prediction for the high-performer sample.....	55
Figure D.2: Prediction for the student-at-risk sample.....	55
Figure D.3: Individual SHAP explanation generated in notebook 03.....	55

LIST OF TABLES

Table 2.1: Summary of reviewed literature.....	4
Table 3.1: Functional requirements.....	7
Table 3.2: Non-functional requirements.....	7
Table 4.1: Technologies, versions and purpose.....	8
Table 5.1: Dataset summary.....	9
Table 5.2: Dataset features, data types and roles.....	10
Table 5.3: Missing values per column.....	10
Table 5.4: Class distribution of the categorical targets (after cleaning).....	11
Table 5.5: Descriptive statistics of numeric columns (after cleaning).....	11
Table 5.6: First five records of the raw dataset (selected columns).....	12
Table 6.1: Data-cleaning report (outputs/reports/data_cleaning_report.json).....	13
Table 6.2: IQR outlier report after cleaning (kept, not removed).....	14
Table 8.1: Model configurations (random_state = 42 throughout).....	21
Table 9.1: Test-set results: academic performance (selected: XGBoost).....	24
Table 9.2: Test-set results: placement readiness (selected: Logistic Regression).....	24
Table 9.3: Regression results: placement probability in percentage points (selected: XGBoost).....	24
Table 9.4: 5-fold cross-validation results on the training set (mean $\pm$ standard deviation).....	25
Table 9.5: Per-class test-set metrics of the selected classifiers.....	26
Table 9.6: Outcomes by gender on the test set (gender is not a model input).....	27
Table 10.1: Top nine features by permutation importance for each selected model.....	29
Table 10.2: SHAP contributions for the average-student example (top eight).....	30
Table 12.1: Project modules and responsibilities.....	35
Table 14.1: Test cases with expected and actual results.....	41
Table 14.2: Streamlit application smoke tests (tests/test_app.py).....	42
Table 15.1: Summary of final model results.....	44
Table 15.2: Predictions for the three sample profiles in the application.....	44
Table 17.1: Possible extensions.....	46
Table A.1: Raw dataset sample, columns 1–10.....	50
Table A.2: Raw dataset sample, columns 11–20.....	50
Table C.1: All metrics: academic performance.....	53
Table C.2: All metrics: placement readiness.....	53
Table C.3: All metrics: placement probability.....	53
Table C.4: Permutation-importance shares of all features.....	53

CHAPTER 1

INTRODUCTION

1.1 Background

Engineering colleges in India invest considerable effort in training and placement activities, and the placement record of a department is one of the most visible indicators of its outcomes. At the same time, the information that describes a student's readiness for campus recruitment is scattered: the CGPA and backlogs sit in the examination system, attendance in departmental registers, coding practice on online judges, and projects, certifications and internships in résumés and portfolios. Faculty mentors therefore often form a picture of a student's preparedness late in the programme, when little time remains to act.

Machine learning (ML) offers a way to combine these heterogeneous indicators into a single, consistent estimate. Educational Data Mining (EDM), a research area surveyed by Romero and Ventura [1] and by Baker and Yacef [2], has shown that student outcomes can be modelled from academic and behavioural data. This project applies standard supervised-learning techniques to that problem and couples them with explainability methods, so that each estimate comes with a statement of which factors raised or lowered it.

1.2 Problem Statement

Given a student's academic record (CGPA, attendance, backlogs, study hours), skill scores (coding/DSA, aptitude, communication, technical skills), experience (projects, certifications, internship, hackathon participation, number of programming languages) and extracurricular score, the system must:

1. classify the student's **academic performance** as Poor, Average, Good or Excellent;
2. classify the student's **placement readiness** as Not Ready, Developing, Ready or Highly Ready;
3. estimate a **placement probability** between 0 % and 100 %; and
4. explain which inputs contributed most to these estimates.

Because real, consented student records containing all of these fields were not available for this academic project, the models are developed and evaluated on a **synthetic dataset** (Chapter 5). The problem is therefore framed as building and validating a complete, reusable ML pipeline and application, not as producing real-world placement evidence.

1.3 Motivation

- **Early intervention.** An estimate available in the third year gives mentors time to recommend internships, projects or coding practice before recruitment begins.
- **Consistency.** A model applies the same criteria to every student, whereas informal judgements vary between mentors.
- **Transparency.** Explainable AI techniques make it possible to show *why* an estimate is low or high, which turns a score into actionable advice.

- **Learning objective.** The project exercises the full ML workflow expected of an AI & ML engineer: data generation and cleaning, leakage-free preprocessing, model comparison, evaluation, explainability, deployment in a web interface and testing.

1.4 Objectives

1. Create a documented dataset of at least 1,000 student records with the required features and three target variables.
2. Build a preprocessing pipeline covering missing values, duplicates, outliers, scaling, encoding, feature selection and a reproducible train/test split without data leakage.
3. Perform exploratory data analysis with labelled, annotated graphs.
4. Train and compare Logistic Regression, Decision Tree, Random Forest, Gradient Boosting and XGBoost classifiers, and suitable regressors for the probability target.
5. Select final models from documented cross-validation results and evaluate them once on a held-out test set.
6. Explain predictions globally (feature importance) and per student (SHAP).
7. Develop a Streamlit application with Dashboard, Student Prediction, Analytics and About sections and full input validation.
8. Test the complete system with documented test cases.

1.5 Scope

In scope: synthetic data generation; data cleaning and preprocessing; EDA; training and comparison of five algorithms for each of the three tasks; model persistence; per-student explanations; a local Streamlit web application; automated tests; and project documentation.

Out of scope: collection of real student data; integration with a university ERP or placement portal; user accounts and authentication; cloud deployment; and any use of the output for admission, ranking or recruitment decisions.

1.6 Proposed Solution

The proposed system (Figure 11.1) is a Python application in which a user enters a student profile through a Streamlit form. After validation, the profile passes through the same scikit-learn preprocessing pipeline used during training, and three saved models produce the academic-performance category, the placement-readiness category and the placement probability. SHAP values are computed for the probability model and presented as a chart of positive and negative contributions, together with the student's main strengths and areas to work on. Dashboard and Analytics pages summarise the dataset and model evaluation.

1.7 Expected Outcome

- A working end-to-end application that returns three explained predictions for any valid student profile, and clear error messages for invalid ones.
- A documented comparison of five algorithms per task, with the selected models justified by cross-validation.
- Reusable artefacts (dataset, trained pipelines, metrics, figures, notebooks, tests and this report) and a clear statement of what the results do and do not show.

CHAPTER 2

LITERATURE REVIEW

2.1 Student Performance Prediction

Predicting academic performance is one of the most studied tasks in EDM. Cortez and Silva [3] used data from two Portuguese secondary schools and compared decision trees, random forests, neural networks and support vector machines; they found that prediction was accurate when earlier period grades were available, and that past grades were the most influential attributes. Kotsiantis et al. [4] studied students in a distance-learning programme and showed that performance could be predicted early in the course from demographic data and the first written assignments. In a systematic review, Shahiri et al. [5] reported that cumulative grade point average and internal assessment marks were the attributes most frequently used in such studies, and decision trees and neural networks the most frequently used methods.

2.2 Educational Data Mining

Romero and Ventura [1] reviewed EDM work and grouped it into tasks such as prediction, clustering, relationship mining and visualisation for instructors. Baker and Yacef [2] described the state of the field in 2009 and identified prediction, clustering, relationship mining, distillation of data for human judgement and discovery with models as its main method categories. Both reviews emphasise that EDM results are meant to support educators' decisions, a principle adopted in this project's interface design.

2.3 Machine Learning in Education

Most applied studies use supervised learning on tabular institutional data. Tree-based ensembles such as Random Forests [8] and gradient-boosted trees [9], [10] are widely used because they capture non-linear relationships and interactions with little feature engineering, while Logistic Regression [12] remains a strong, interpretable baseline. A recurring methodological risk is *data leakage* — information from the evaluation data influencing training — which Kaufman et al. [16] showed can produce misleadingly optimistic results. This project addresses leakage explicitly (Section 6.8).

2.4 Placement Prediction

Placement prediction applies the same techniques to recruitment outcomes. Maurya et al. [6] developed and compared several supervised classifiers for predicting engineering students' placement from academic performance data. Kumar et al. [7] likewise investigated ML algorithms for predicting college students' placements from academic performance. Such studies typically predict a binary placed/not-placed label from historical institutional records; this project instead predicts a four-level readiness category and a continuous probability, and adds per-student explanations.

2.5 Classification Algorithms

The algorithms compared in this project are standard, well-documented methods: Logistic Regression [12], decision trees in the CART family [11], Random Forests [8], gradient boosting [9] and its scalable, regularised implementation XGBoost [10]. Their principles are described in Chapter 8. All are available through scikit-learn [17] or the XGBoost library, which makes the comparison reproducible.

2.6 Explainable AI

When predictions affect people, it is important to understand how a model reaches them [18]. Local, model-agnostic explanation methods include LIME [15], which fits a simple surrogate model around one prediction, and SHAP [14], which attributes a prediction to features using Shapley values from cooperative game theory and unifies several earlier additive attribution methods. SHAP is used in this project because its contributions add up exactly to the difference between a prediction and the average prediction, which makes them easy to present in percentage points. Fairness concerns — for example, whether a model treats demographic groups differently — are discussed by Barocas et al. [19].

2.7 Summary of Reviewed Literature

Table 2.1: Summary of reviewed literature

Paper / Study	Year	Method	Dataset / Context	Key Finding	Relevance to this project
Kotsiantis, Pierrakeas & Pintelas [4]	2004	Six supervised learners compared	Distance-learning students (Hellenic Open University)	Performance can be predicted early from demographic and early assignment data	Supports early-warning use of predictions
Cortez & Silva [3]	2008	DT, RF, NN, SVM	Two Portuguese secondary schools	Earlier grades are the strongest predictors; good accuracy when available	Justifies CGPA as a key academic feature
Baker & Yacef [2]	2009	Review	EDM field	Defines main EDM method categories	Places this project in the prediction category
Romero & Ventura [1]	2010	Review	EDM literature	EDM should support educators' decisions	Motivates advisory, not decision-making, design
Kaufman et al. [16]	2012	Formal analysis of leakage	Data-mining competitions and practice	Leakage inflates reported performance	Leakage-free Pipeline design (Section 6.8)
Shahiri, Husain & Rashid [5]	2015	Systematic review	Student-performance studies	CGPA and internal marks most used; DT and NN most common	Guides feature choice and baseline models
Chen & Guestrin [10]	2016	XGBoost	Benchmarks and competitions	Scalable, regularised gradient boosting	One of the five compared models
Lundberg & Lee [14]	2017	SHAP	Model explanation	Unified additive attributions based on Shapley values	Per-student explanations in the app

Paper / Study	Year	Method	Dataset / Context	Key Finding	Relevance to this project
Maurya, Hussain & Singh [6]	2021	Several supervised classifiers	Engineering students' academic records	Placement can be predicted from academic performance	Direct precedent for placement prediction
Kumar et al. [7]	2023	ML classifiers	College students' academic performance data	ML approaches applied to placement prediction	Direct precedent; binary target vs. our 4 levels

The reviewed work establishes that academic and skill indicators carry predictive information about student outcomes, and that tree ensembles and logistic regression are appropriate baselines. The gap addressed here is educational rather than scientific: an end-to-end, explainable and tested implementation suitable for a mini-project, with honest reporting of its synthetic-data limitation.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

In a typical department, placement preparedness is judged informally by mentors and the training and placement cell using CGPA cut-offs, mock-test scores and personal knowledge of students. The limitations of this approach are:

- **Fragmented data.** Academic, skill and experience indicators are stored in different places and are rarely viewed together.
- **Single-criterion screening.** CGPA or backlog cut-offs ignore coding ability, projects and internships.
- **Late identification.** Weak preparedness is often discovered during the recruitment season itself.
- **Inconsistency.** Different mentors weigh the same evidence differently.
- **No explanation.** A cut-off tells a student that they are below a threshold, not what would help most.

3.2 Proposed System

The proposed system combines 14 academic, skill and experience features in trained ML models that output three estimates and an explanation. Its components are a synthetic data generator, a leakage-free preprocessing pipeline, a training script that compares five algorithms per task, saved model pipelines, a prediction module with SHAP explanations, a Streamlit application and an automated test suite.

3.3 Advantages

- **Multi-factor.** Uses 14 model features instead of a single cut-off.
- **Validated selection.** Every model is chosen by 5-fold cross-validation on 960 training records and evaluated on 240 held-out records (Chapter 9).
- **Explained output.** Each prediction lists the contribution of every feature in percentage points (Chapter 10).
- **Fast.** A prediction including SHAP explanation took 26.3 ms on average (95th percentile 27.2 ms) over 50 runs in the development environment.
- **Input safety.** Every field is range- and type-checked before any model runs (Chapter 14).
- **Demographic exclusion.** Age and gender are not model inputs; a test confirms that changing gender does not change the prediction.
- **Reproducible.** A fixed random seed (42) regenerates the identical dataset and the same selected models.

3.4 Limitations

- The dataset is synthetic; results show that the pipeline recovers the simulated relationships, not that it predicts real placements.

- The three models are trained independently, so borderline students may receive a readiness category and a probability that appear slightly inconsistent.
- Skill scores such as communication are simplified single numbers.
- The test set contains 240 records, and some groups (e.g. gender 'Other', 6 test records) are very small.

3.5 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	The system shall load the dataset and display total students, average CGPA, average attendance, average coding score and the placement-readiness distribution.
FR2	The system shall accept 16 inputs: age, gender, CGPA, attendance, backlogs, coding, aptitude, communication, projects, certifications, internship, hackathon, extracurricular, technical skills, programming languages and study hours.
FR3	The system shall validate every input and display a specific error message for each invalid field.
FR4	On pressing PREDICT, the system shall display academic performance, placement readiness and estimated placement probability.
FR5	The system shall display the main contributing factors of each prediction.
FR6	The system shall provide interactive analytics charts and feature importance.
FR7	The system shall display model-comparison tables and confusion matrices.
FR8	The system shall provide an About section describing objective, dataset, algorithms, technologies, limitations and ethics.
FR9	The training script shall regenerate all models, metrics and figures from the dataset.
FR10	The system shall offer sample student profiles to prefill the form.

3.6 Non-functional Requirements

Table 3.2: Non-functional requirements

Category	Requirement	How it is met
Performance	A prediction should return within one second.	Measured mean 26.3 ms per explained prediction; models are loaded once and cached (st.cache_resource).
Usability	A first-time user should be able to obtain a prediction without instructions.	Grouped form (Academics, Skills, Experience & profile), sample profiles, one prominent PREDICT button, plain-language errors.
Reliability	Invalid or missing input must never reach the model.	validate_input() runs before prediction and again inside StudentPredictor; 18 documented test cases.
Maintainability	Code should be modular and reproducible.	Separate modules for data, preprocessing, training, prediction and utilities; fixed seed; single command retraining.
Security & privacy	No personal data should be collected or stored.	Runs locally; inputs are not saved; dataset is synthetic; age and gender are not used by the models.

CHAPTER 4

TECHNOLOGY USED

The project uses the Python scientific stack. Table 4.1 lists every technology, the version used and its purpose; the paragraphs below explain how the central libraries fit together.

Table 4.1: Technologies, versions and purpose

Technology	Version	Purpose and use in this project
Python [21]	3.12 (3.11+ supported)	Implementation language; the project is a package (src/) plus a Streamlit entry point (app.py)
pandas [23], [31]	3.0.2	Reads the CSV, removes duplicates, reports missing values, computes descriptive statistics
NumPy [26]	2.4.4	Numerical arrays; seeded random generator used to create the synthetic dataset
scikit-learn [17], [20]	1.8.0	Pipeline, ColumnTransformer, SimpleImputer, StandardScaler, OneHotEncoder, VarianceThreshold; Logistic Regression, Decision Tree, Random Forest, Gradient Boosting; StratifiedKFold cross-validation; permutation importance; all metrics
XGBoost [10], [24]	3.4.1	Regularised gradient-boosted tree classifier and regressor (selected for two of three tasks)
SHAP [14], [25]	0.52.0	Per-prediction explanations: TreeExplainer for tree models, LinearExplainer for linear models; global summary plots
Matplotlib [27]	3.10.8	Static figures in notebooks, training script and this report
Seaborn [28]	0.13.2	Statistical plots: histograms, box plots, heatmaps, confusion matrices
Plotly [29]	7.1.0	Interactive charts in the application (hover for exact values)
Streamlit [22]	1.64.0	Web interface: forms, inputs, tabs, metrics, charts; caching so data and models load once
Joblib [32]	1.5.3	Serialises fitted pipelines to models/*.pkl so the app never retrains
Jupyter Notebook [30]	nbconvert 7.17.1	Three executed notebooks: data generation, EDA, model training
pytest	9.1.1	Automated tests of predictions and of the Streamlit app

The libraries form a single chain. pandas and NumPy prepare the data; scikit-learn wraps preprocessing and every model in one Pipeline object so that the same transformations are applied during training, cross-validation and prediction; XGBoost plugs into that Pipeline through its scikit-learn-compatible interface. Joblib saves the fitted Pipelines, which the Streamlit application loads to make predictions, while SHAP explains them. Matplotlib and Seaborn produce the static figures in this report, and Plotly the interactive charts inside the application. Jupyter notebooks document the experiments step by step, and pytest verifies the finished system.

CHAPTER 5

DATASET

5.1 Nature of the Dataset

The dataset used in this academic project is synthetic and was generated for demonstration and experimentation. Therefore, model performance should not be interpreted as evidence of real-world placement outcomes.

A public dataset containing all required fields (academic record, coding and aptitude scores, projects, internships, hackathons and placement outcomes) was not available, and real student records are personal data. The dataset was therefore generated by `src/generate_data.py` with a fixed seed of 42. Each simulated student has two hidden traits, *ability* and *diligence*, drawn from a standard normal distribution; observable features are noisy functions of these traits, so they correlate in plausible ways. The three targets are computed from observable features plus random noise:

- **Academic performance index** — mainly CGPA, attendance, backlogs and study hours — binned into Poor, Average, Good and Excellent.
- **Placement score** — coding, aptitude, communication and technical-skills scores, CGPA, projects, certifications, internship, hackathon, extracurricular score, attendance and backlogs — binned into Not Ready, Developing, Ready and Highly Ready.
- **Placement probability** — a logistic transform of the placement score plus noise, limited to 0.5–99.5 %.

To exercise the preprocessing pipeline, the generator deliberately injects data-quality problems: about 2 % missing values in six columns, 15 exact duplicate rows and six impossible values.

5.2 Dataset Size and Structure

Table 5.1: Dataset summary

Property	Value
File	data/student_dataset.csv
Raw records	1,215 (1,200 unique students + 15 duplicates)
Columns	20 (1 identifier, 2 demographic, 14 model features, 3 targets)
Records after cleaning	1,200
Training / test records	960 / 240 (80 / 20, stratified by placement readiness)
Missing values (raw)	148 cells in 6 columns

5.3 Feature Description

Table 5.2: Dataset features, data types and roles

Column	Pandas type	Role	Valid range / values	Description
student_id	text	Identifier	STU00001 ...	Unique student code (not used for modelling)
age	integer	Demographic	19–24 in data	Age in years (not a model input)
gender	text	Demographic	Male / Female / Other	Recorded for fairness audit only
cgpa	float	Feature	0–10	Cumulative grade point average
attendance_pct	float	Feature	0–100	Attendance percentage
backlogs	integer	Feature	≥ 0	Number of uncleared subjects
coding_score	float	Feature	0–100	Coding / DSA assessment score
aptitude_score	integer	Feature	0–100	Quantitative and logical aptitude
communication_score	float	Feature	0–100	Communication assessment
projects	integer	Feature	≥ 0	Number of completed projects
certifications	float	Feature	≥ 0	Number of certifications
internship	text	Feature	Yes / No	Internship experience
hackathon	text	Feature	Yes / No	Hackathon participation
extracurricular_score	float	Feature	0–100	Extracurricular involvement
technical_skills_score	integer	Feature	0–100	Overall technical skills
programming_languages	integer	Feature	≥ 0	Number of languages known
study_hours_per_week	float	Feature	0–100	Self-study hours per week
academic_performance	text	Target 1	4 classes	Poor / Average / Good / Excellent
placement_readiness	text	Target 2	4 classes	Not Ready / Developing / Ready / Highly Ready
placement_probability	float	Target 3	0–100	Estimated placement probability (%)

Columns containing injected missing values or impossible entries are stored by pandas as floating-point numbers even where the concept is an integer (e.g. certifications), because NaN is a float value.

5.4 Missing Values

Table 5.3: Missing values per column

Column	Missing in raw file	Share of raw rows	Missing after cleaning
attendance_pct	26	2.14 %	26
communication_score	24	1.98 %	24
certifications	24	1.98 %	24
internship	24	1.98 %	24
extracurricular_score	25	2.06 %	24

Column	Missing in raw file	Share of raw rows	Missing after cleaning
study_hours_per_week	25	2.06 %	25
cgpa / coding_score	0	0.00 %	2 / 1 (impossible values set to NaN)

5.5 Class Distribution

Table 5.4: Class distribution of the categorical targets (after cleaning)

Academic performance	Students	Share	Placement readiness	Students	Share
Poor	338	28.2 %	Not Ready	420	35.0 %
Average	370	30.8 %	Developing	228	19.0 %
Good	316	26.3 %	Ready	184	15.3 %
Excellent	176	14.7 %	Highly Ready	368	30.7 %
Total	1200	100 %	Total	1200	100 %

Both targets are imbalanced: only 14.7 % of students are *Excellent* and 15.3 % are *Ready*. Models are therefore compared with macro-averaged F1, which weights every class equally. The placement probability has mean 47.5 % (Table 5.5).

5.6 Descriptive Statistics

Table 5.5: Descriptive statistics of numeric columns (after cleaning)

Feature	Count	Mean	Std	Min	25 %	Median	75 %	Max
cgpa	1198	6.94	0.97	4.00	6.33	6.96	7.56	10.00
attendance_pct	1174	78.52	9.47	48.50	72.30	78.60	84.90	100.00
backlogs	1200	0.77	1.56	0.00	0.00	0.00	1.00	10.00
coding_score	1199	57.32	16.42	5.00	46.00	57.00	69.00	100.00
aptitude_score	1200	59.29	14.14	10.00	50.00	59.00	68.00	100.00
communication_score	1176	64.01	12.71	21.00	56.00	64.00	73.00	100.00
projects	1200	1.79	1.66	0.00	1.00	1.00	3.00	10.00
certifications	1176	1.61	1.57	0.00	0.00	1.00	2.00	9.00
extracurricular_score	1176	57.94	15.34	12.00	48.00	57.00	68.00	100.00
technical_skills_score	1200	46.20	16.52	5.00	35.00	45.00	57.00	100.00
programming_languages	1200	2.73	1.53	1.00	2.00	2.00	4.00	8.00
study_hours_per_week	1175	13.89	6.67	1.00	9.40	13.80	18.20	35.70
age	1200	20.94	1.15	19.00	20.00	21.00	22.00	24.00
placement_probability	1200	47.45	37.23	0.50	8.90	42.20	88.82	99.50

Counts below 1,200 reflect remaining missing values that are imputed inside the training pipeline. Because impossible values have been removed, all maxima and minima lie within the valid ranges.

5.7 Representative Records

Table 5.6: First five records of the raw dataset (selected columns)

ID	CGPA	Att. %	Backlogs	Coding	Projects	Intern.	Academic	Readiness	Prob. %
STU00740	4.87	72.0	3	30.0	0	No	Poor	Not Ready	2.1
STU00789	5.82	80.3	1	43.0	0	No	Poor	Not Ready	5.6
STU00044	7.56	75.8	0	68.0	1	No	Good	Developing	47.7
STU00156	9.23	90.1	0	85.0	5	Yes	Excellent	Highly Ready	99.5
STU00495	6.35	75.4	2	51.0	1	No	Poor	Not Ready	7.7

The full set of columns for the first six records is given in Appendix A. A dash (—) marks a missing value.

CHAPTER 6

DATA PREPROCESSING

Preprocessing is divided into two stages to prevent data leakage (Figure 6.1). Stage 1 applies fixed rules that learn nothing from the data and can therefore safely run before the train/test split. Stage 2 contains every step that learns statistics; it is packaged in a scikit-learn Pipeline and fitted only on training data.

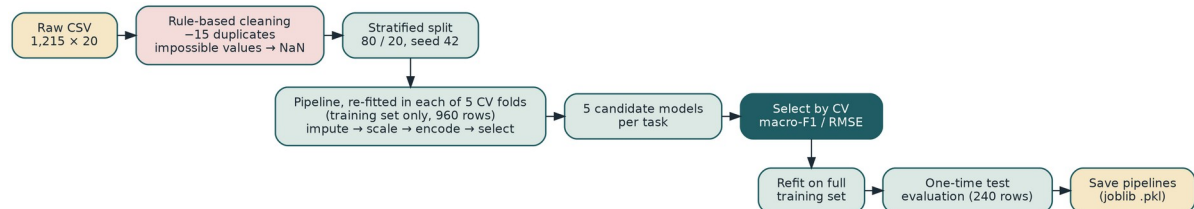


Figure 6.1: End-to-end ML pipeline

The rule-based cleaning runs on the whole file; everything that learns from data runs inside the cross-validation loop on the training set only. The test set is used once, after selection.

6.1 Data Cleaning

`clean_dataset()` in `src/preprocessing.py` received 1215 rows and returned 1200. Table 6.1 lists what it did.

Table 6.1: Data-cleaning report (`outputs/reports/data_cleaning_report.json`)

Step	Rule	Result
Duplicate removal	Drop exact duplicate rows	15 rows removed
Missing targets	Drop rows without a target value	0 rows removed
Impossible values	Values outside the valid domain → NaN	cgpa: 2, attendance_pct: 2, coding_score: 1, study_hours_per_week: 1

6.2 Missing-value Handling

Missing numeric values are replaced by the median of the training fold (`SimpleImputer(strategy="median")`), which is robust to skewed distributions such as backlogs. The missing internship values are replaced by the most frequent category of the training fold. Figure 6.2 shows the missing values in the raw file.

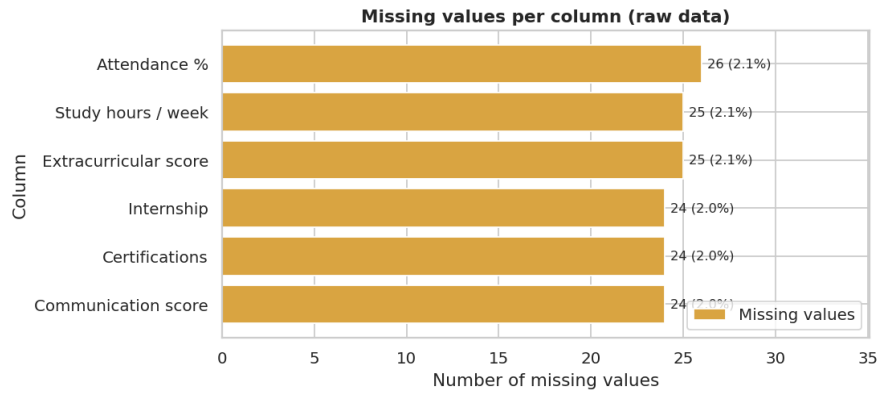


Figure 6.2: Missing values per column in the raw data

Six columns contain about 2 % missing values each; the remaining columns are complete.

6.3 Duplicate Handling

The 15 injected duplicates are exact copies of existing rows (a record uploaded twice). They were removed before splitting; otherwise a record could appear in both the training and test sets, which would inflate test performance.

6.4 Outlier Handling

Two kinds of outliers are distinguished. **Impossible values** (CGPA 12.4 and -1 , attendance 134 % and 250 %, coding score -8 , 160 study hours per week) are data-entry errors; they are set to NaN and imputed. **Statistical outliers** inside the valid domain are real students and are kept. They were identified with the interquartile-range (IQR) rule, flagging values below $Q1 - 1.5 \cdot IQR$ or above $Q3 + 1.5 \cdot IQR$.

Table 6.2: IQR outlier report after cleaning (kept, not removed)

Feature	Q1	Q3	Lower fence	Upper fence	Outliers	Share
cgpa	6.33	7.56	4.49	9.40	15	1.25 %
attendance_pct	72.30	84.90	53.40	103.80	3	0.26 %
backlogs	0.00	1.00	-1.50	2.50	103	8.58 %
coding_score	46.00	69.00	11.50	103.50	3	0.25 %
aptitude_score	50.00	68.00	23.00	95.00	17	1.42 %
communication_score	56.00	73.00	30.50	98.50	7	0.60 %
projects	1.00	3.00	-2.00	6.00	19	1.58 %
certifications	0.00	2.00	-3.00	5.00	28	2.38 %
extracurricular_score	48.00	68.00	18.00	98.00	14	1.19 %
technical_skills_score	35.00	57.00	2.00	90.00	8	0.67 %
programming_languages	2.00	4.00	-1.00	7.00	10	0.83 %
study_hours_per_week	9.40	18.20	-3.80	31.40	3	0.26 %

Backlogs have the most IQR outliers (103, 8.58 %) because most students have none, so any student with three or more backlogs lies above the upper fence. These are genuine and informative cases, so removing them would bias the models.

6.5 Encoding

The two categorical features, internship and hackathon, take the values Yes and No. They are one-hot encoded with `OneHotEncoder(drop="if_binary")`, which produces a single 0/1 column for each (`internship_Yes`, `hackathon_Yes`). The target categories are encoded as ordered integers 0–3 for training and decoded back to labels for display.

6.6 Feature Scaling

Numeric features are standardised to zero mean and unit variance with `StandardScaler`, fitted on the training fold. Scaling matters for Logistic Regression and Ridge Regression, whose regularisation penalises coefficients on the same scale; tree-based models are unaffected by it.

6.7 Feature Engineering and Selection

No derived features were added, so that every model input corresponds to a field the user enters. Feature selection was applied in two ways:

- **By design:** `student_id` carries no information, and age and gender are excluded to avoid demographic bias (Section 16.4). The `ColumnTransformer` drops these columns (`remainder="drop"`).
- **Automatically:** a `VarianceThreshold` step removes constant or near-constant features. All 14 features passed this check, giving the final input vector: `cgpa`, `attendance_pct`, `backlogs`, `coding_score`, `aptitude_score`, `communication_score`, `projects`, `certifications`, `extracurricular_score`, `technical_skills_score`, `programming_languages`, `study_hours_per_week`, `internship_Yes`, `hackathon_Yes`.

6.8 Train/Test Split and Prevention of Data Leakage

The cleaned data was split 80/20 into 960 training and 240 test records with `train_test_split(stratify=placement_readiness, random_state=42)`, preserving class proportions. Leakage is prevented by four measures:

1. Rule-based cleaning before the split learns no statistics from the data.
2. Imputation, scaling, encoding and variance filtering are steps of a Pipeline, so `cross_validate` re-fits them inside every fold.
3. Model selection uses only cross-validation scores on the training set.
4. The test set is used once, after selection, to report final metrics.

```
def build_preprocessor() -> Pipeline:
    numeric = Pipeline([("impute", SimpleImputer(strategy="median")),
                        ("scale", StandardScaler())])
    binary = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("encode", OneHotEncoder(drop="if_binary",
                                                categories=[["No", "Yes"]] * 2))
    ])
    columns = ColumnTransformer(
        [("num", numeric, NUMERIC_FEATURES), ("bin", binary, BINARY_FEATURES)],
        remainder="drop")      # drops student_id, age, gender and targets
    return Pipeline([("columns", columns),
                    ("select", VarianceThreshold(threshold=1e-4))])
```

Listing 6.1: The preprocessing pipeline (src/preprocessing.py)

CHAPTER 7

EXPLORATORY DATA ANALYSIS

All figures in this chapter were produced by notebooks/02_eda.ipynb or computed from data/student_dataset.csv after cleaning (1,200 records). Because the data is synthetic, the patterns observed here confirm that the generator behaves as designed; they are not discoveries about real students.

7.1 Dataset Distribution

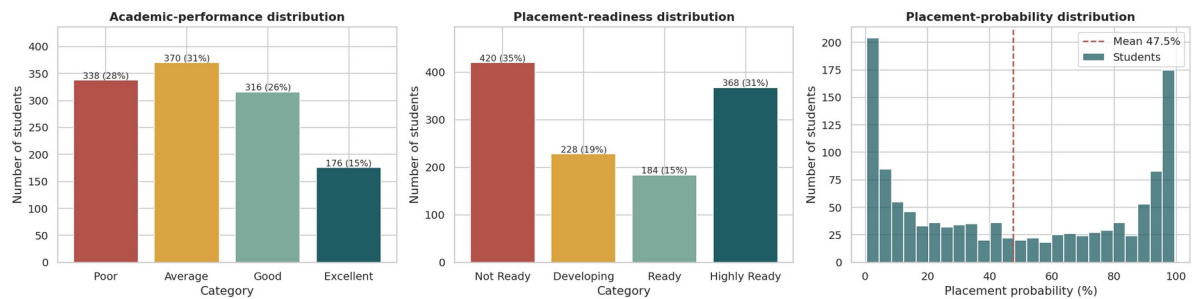


Figure 7.1: Distribution of the three target variables

Academic performance is dominated by Average (370) and Poor (338); placement readiness is bimodal, with Not Ready (420) and Highly Ready (368) the largest classes; placement probability is spread across the full range with a mean of 47.5 %.

7.2 Univariate Distributions

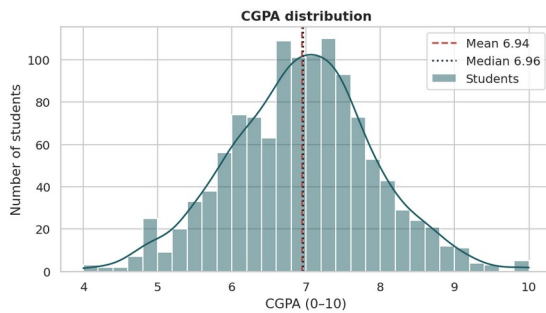


Figure 7.2: CGPA distribution

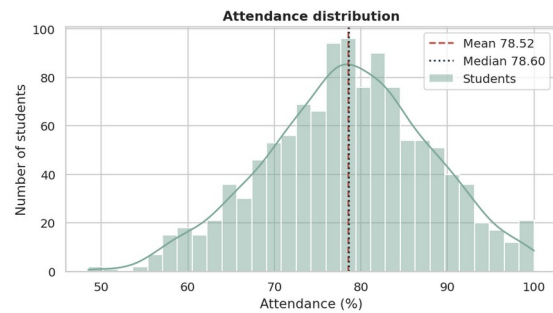


Figure 7.3: Attendance distribution

CGPA is approximately normal with mean 6.94 and median 6.96, ranging from 4.00 to 10.00. Attendance has mean 78.5 % and median 78.60 %, with a slight left tail down to 48.5 %.

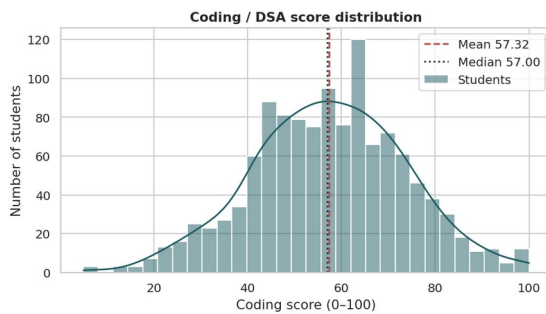


Figure 7.4: Coding / DSA score distribution

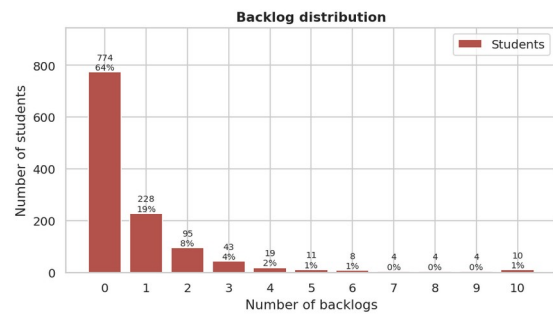


Figure 7.5: Backlog distribution

Coding scores have mean 57.3 and standard deviation 16.42, the widest spread of the skill scores. Backlogs are strongly right-skewed: most students have none and a small group has several.

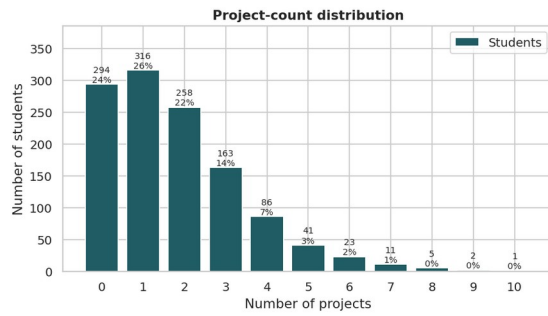


Figure 7.6: Project-count distribution

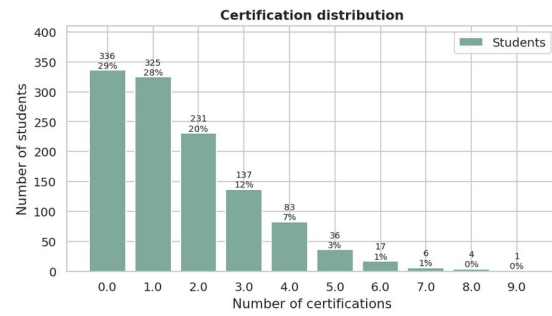


Figure 7.7: Certification distribution

Most students have completed one to three projects (median one); certifications follow a similar count distribution with median one.

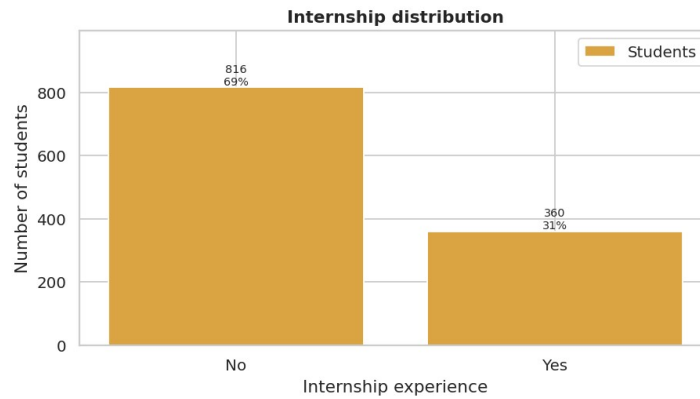


Figure 7.8: Internship distribution

360 students (30 %) have internship experience and 816 do not; 24 values are missing.

7.3 Correlation Analysis

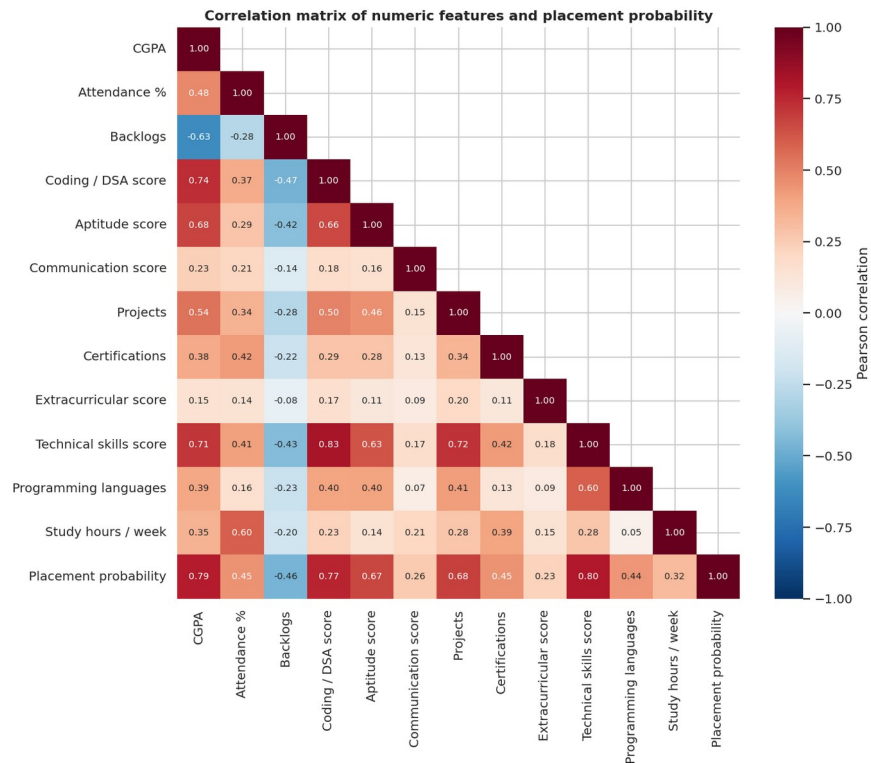


Figure 7.9: Correlation heatmap of numeric features and placement probability

Placement probability correlates most strongly with technical skills ($r = 0.797$), CGPA (0.787) and coding score (0.765), and negatively with backlogs (-0.461). Attendance and CGPA are positively correlated because both depend on the hidden diligence trait.

7.4 Relationships with Placement Readiness

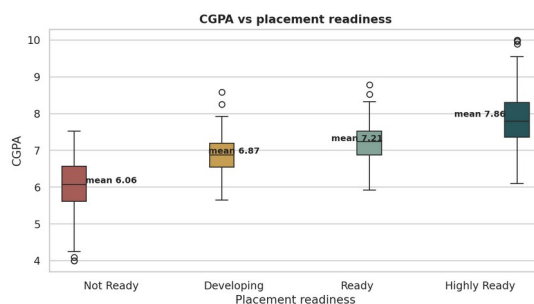


Figure 7.10: CGPA vs placement readiness

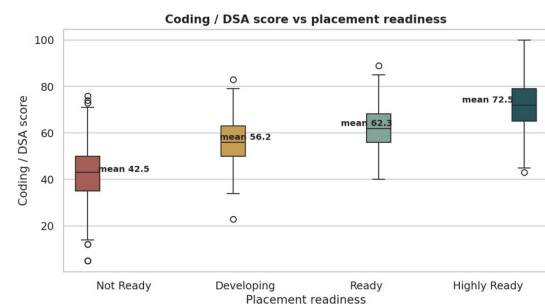


Figure 7.11: Coding score vs placement readiness

Mean CGPA rises from 6.06 (Not Ready) through 6.87 and 7.21 to 7.86 (Highly Ready), with substantial overlap between neighbouring classes. Mean coding score rises from 42.5 to 72.46, the clearest separation of any single feature.

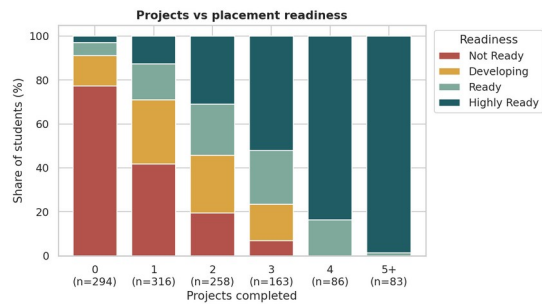


Figure 7.12: Projects vs placement readiness

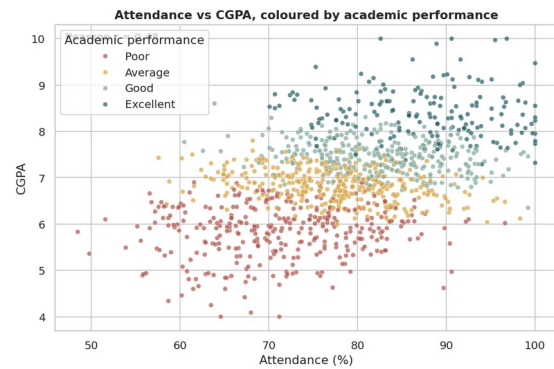


Figure 7.13: Attendance vs CGPA by academic performance

The share of Highly Ready students increases steadily with the number of projects (mean 0.63 projects for Not Ready vs 3.33 for Highly Ready). Higher attendance is associated with higher CGPA, and the academic categories form bands along the CGPA axis.

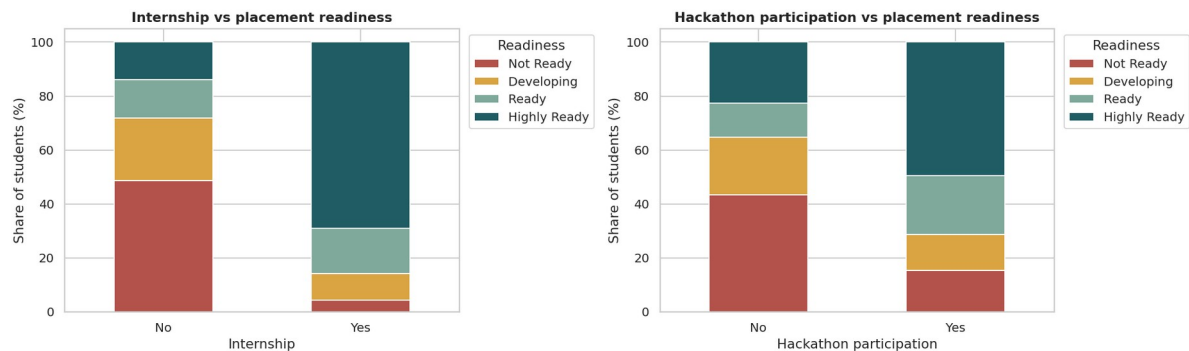


Figure 7.14: Internship and hackathon participation vs placement readiness

69.2 % of students with an internship are Highly Ready, compared with 13.8 % of those without; 48.7 % of students without an internship are Not Ready.

7.5 Feature Importance

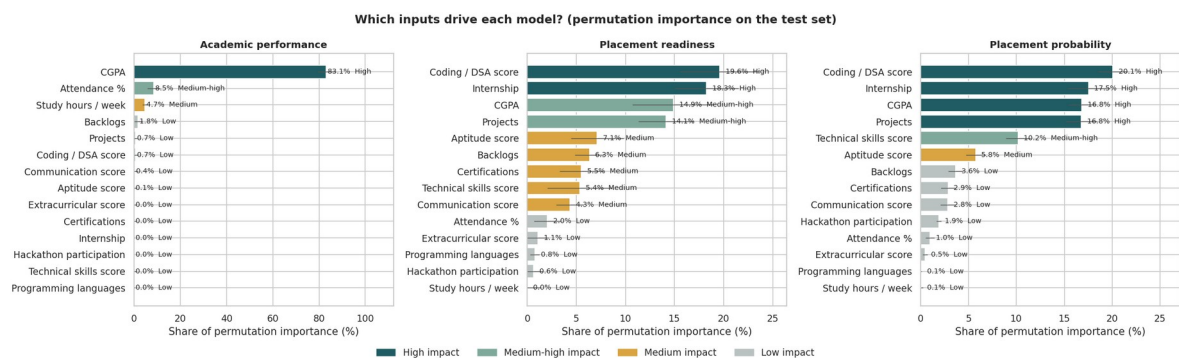


Figure 7.15: Permutation feature importance of the three selected models

Coding score, internship, CGPA and projects dominate both placement models, while academic performance depends almost entirely on CGPA, attendance and study hours. Class imbalance makes macro-F1 the right primary metric for the comparisons in Chapter 9; Chapter 10 discusses importance in detail.

CHAPTER 8

MACHINE LEARNING METHODOLOGY

Three supervised learning tasks are solved: two four-class classification problems (academic performance, placement readiness) and one regression problem (placement probability). Five algorithm families were tested for each task. Table 8.1 lists the exact configurations used.

Table 8.1: Model configurations (random_state = 42 throughout)

Model	Classification configuration	Regression configuration
Logistic Regression / Ridge	LogisticRegression(C=1.0, max_iter=2000)	Ridge(alpha=1.0)
Decision Tree	max_depth=6, min_samples_leaf=10	max_depth=6, min_samples_leaf=10
Random Forest	n_estimators=300, min_samples_leaf=3	n_estimators=300, min_samples_leaf=3
Gradient Boosting	n_estimators=200, learning_rate=0.05, max_depth=3	n_estimators=300, learning_rate=0.05, max_depth=3
XGBoost	n_estimators=300, learning_rate=0.05, max_depth=3, subsample=0.9, colsample_bytree=0.9	n_estimators=400, same other settings

8.1 Logistic Regression

Working principle. Logistic Regression [12] models the probability of each class as a function of a weighted sum of the inputs. For more than two classes, scikit-learn fits a multinomial model in which a softmax function converts one linear score per class into probabilities. Coefficients are learned by maximising the regularised likelihood. For the probability target, its regression counterpart, Ridge Regression, fits a linear model with an L2 penalty.

Advantages. Fast, stable with small datasets, produces calibrated-looking probabilities and has directly interpretable coefficients.

Limitations. Can only represent linear decision boundaries in the input space unless features are transformed; sensitive to feature scaling.

Why it was tested. It is the standard interpretable baseline. It was expected to do well on placement readiness because the synthetic placement score is close to a linear combination of the inputs.

8.2 Decision Tree

Working principle. A decision tree [11] recursively splits the data on the feature and threshold that most reduce impurity (Gini impurity for classification, squared error for regression), producing a tree of if-then rules. Depth was limited to 6 and each leaf required at least 10 samples to limit overfitting.

Advantages. Easy to visualise and explain; handles non-linear relationships and needs no scaling.

Limitations. High variance: small changes in data can produce a very different tree; piecewise-constant predictions approximate smooth relationships poorly.

Why it was tested. It is the building block of the ensemble methods and a commonly used method in EDM studies [5].

8.3 Random Forest

Working principle. A Random Forest [8] trains many decision trees on bootstrap samples of the data, considering a random subset of features at each split, and averages their predictions (or votes). 300 trees were used.

Advantages. Much lower variance than a single tree; robust to noise and outliers; little tuning required.

Limitations. Less interpretable than a single tree; larger model files; can still struggle with smooth, near-linear relationships.

Why it was tested. A strong, widely used general-purpose baseline for tabular data.

8.4 Gradient Boosting

Working principle. Gradient boosting [9] builds trees sequentially; each new shallow tree (depth 3) is fitted to the negative gradient of the loss with respect to the current ensemble's predictions, and its contribution is scaled by the learning rate (0.05).

Advantages. Often among the most accurate methods for tabular data; captures interactions.

Limitations. More sensitive to hyperparameters; trained sequentially, so slower than Random Forest; can overfit without shrinkage.

Why it was tested. Represents the boosting family using scikit-learn's reference implementation.

8.5 XGBoost

Working principle. XGBoost [10] is an optimised gradient-boosting library that adds L1/L2 regularisation of leaf weights, second-order gradient information, row and column subsampling (0.9 each here) and efficient split finding.

Advantages. High accuracy, built-in regularisation, fast training.

Limitations. More hyperparameters than the other models; less transparent without tools such as SHAP.

Why it was tested. Required by the project specification where appropriate, and a state-of-the-art method for tabular prediction.

8.6 Training, Validation and Prediction

8.6.1 Training

Each model is wrapped in `Pipeline([("preprocess", build_preprocessor()), ("model", estimator)])`, so preprocessing and model are trained together. After selection, the chosen pipeline is refitted on all 960 training records.

8.6.2 Validation

Models are compared with 5-fold cross-validation on the training set: stratified folds for classification (`StratifiedKFold(5, shuffle=True, random_state=42)`) and ordinary folds for regression. The selection rule is fixed in advance: highest mean cross-validated **macro-F1** for classification, lowest mean cross-validated **RMSE** for regression. The test set plays no part in selection.

8.6.3 Prediction

For a new student, the saved pipeline applies the fitted preprocessing and returns class probabilities; the predicted category is the most probable class, and the model confidence shown in the app is that class's probability. The regression output is clipped to 1–99 % because the model has no basis for absolute certainty.

8.6.4 Hyperparameter Tuning

No automated hyperparameter search was performed. Each model uses the fixed, conservative configuration in Table 8.1 (for example limited tree depth and a small learning rate), chosen to reduce overfitting on a dataset of this size. The same configurations were used for every task so that the comparison is fair. Systematic tuning with `GridSearchCV` is listed as future work.

CHAPTER 9

MODEL EVALUATION

This chapter reports the actual results written by `src/train.py` to `outputs/metrics/`. Cross-validation (CV) figures are means over five folds of the 960-record training set; test figures come from a single evaluation on the 240-record held-out test set. Precision, recall and F1 are macro-averaged over the four classes; ROC-AUC is the macro-averaged one-vs-rest AUC. The selected model in each table is highlighted.

9.1 Academic Performance Classification

Table 9.1: Test-set results: academic performance (selected: XGBoost)

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
XGBoost	0.800	0.814	0.810	0.812	0.959
Logistic Regression	0.817	0.829	0.821	0.824	0.967
Random Forest	0.783	0.809	0.787	0.797	0.953
Gradient Boosting	0.787	0.804	0.797	0.799	0.959
Decision Tree	0.767	0.797	0.777	0.785	0.934

9.2 Placement Readiness Classification

Table 9.2: Test-set results: placement readiness (selected: Logistic Regression)

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.867	0.842	0.829	0.835	0.975
XGBoost	0.796	0.755	0.756	0.754	0.947
Gradient Boosting	0.771	0.719	0.721	0.719	0.943
Random Forest	0.804	0.757	0.750	0.751	0.946
Decision Tree	0.713	0.658	0.651	0.651	0.882

9.3 Placement Probability Regression

Table 9.3: Regression results: placement probability in percentage points (selected: XGBoost)

Model	CV RMSE ($\pm$ std)	CV R ²	Test MAE	Test RMSE	Test R ²
XGBoost	10.35 $\pm$ 0.66	0.922	7.07	9.87	0.930
Gradient Boosting	10.59 $\pm$ 0.70	0.918	7.40	10.39	0.923
Random Forest	11.48 $\pm$ 0.86	0.904	7.95	11.21	0.910
Ridge Regression	14.76 $\pm$ 0.86	0.842	10.01	12.78	0.883
Decision Tree	15.80 $\pm$ 0.62	0.818	9.98	14.67	0.846

MAE and RMSE are in percentage points: the selected XGBoost model's estimates differ from the synthetic target by 7.07 points on average.

9.4 Cross-validation Results and Model Selection

Table 9.4: 5-fold cross-validation results on the training set (mean $\pm$ standard deviation)

Task	Model	CV macro-F1	CV accuracy	CV ROC-AUC
Academic	XGBoost	0.836 $\pm$ 0.025	0.834 $\pm$ 0.023	0.963 $\pm$ 0.006
Academic	Logistic Regression	0.831 $\pm$ 0.016	0.833 $\pm$ 0.017	0.969 $\pm$ 0.005
Academic	Random Forest	0.826 $\pm$ 0.024	0.826 $\pm$ 0.023	0.957 $\pm$ 0.006
Academic	Gradient Boosting	0.819 $\pm$ 0.030	0.817 $\pm$ 0.029	0.957 $\pm$ 0.011
Academic	Decision Tree	0.802 $\pm$ 0.032	0.801 $\pm$ 0.031	0.937 $\pm$ 0.008
Readiness	Logistic Regression	0.781 $\pm$ 0.029	0.823 $\pm$ 0.020	0.963 $\pm$ 0.008
Readiness	XGBoost	0.710 $\pm$ 0.018	0.765 $\pm$ 0.017	0.936 $\pm$ 0.002
Readiness	Gradient Boosting	0.694 $\pm$ 0.023	0.754 $\pm$ 0.025	0.926 $\pm$ 0.005
Readiness	Random Forest	0.691 $\pm$ 0.040	0.760 $\pm$ 0.024	0.934 $\pm$ 0.003
Readiness	Decision Tree	0.621 $\pm$ 0.019	0.692 $\pm$ 0.015	0.862 $\pm$ 0.012

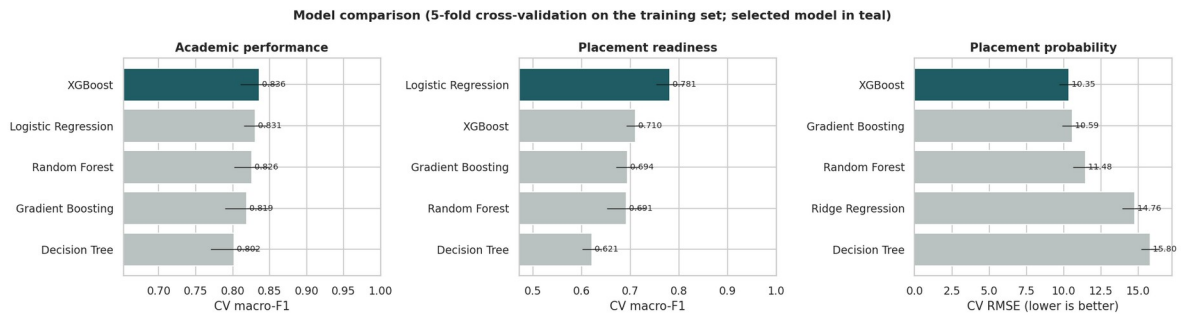


Figure 9.1: Model comparison by cross-validation score (selected model in teal)

Error bars show the standard deviation across the five folds.

Academic performance. XGBoost achieved the highest mean CV macro-F1 (0.836) and was selected. Logistic Regression was very close (0.831), within one fold-to-fold standard deviation (± 0.025), and scored slightly higher on the test set (macro-F1 0.824 vs 0.812). This is normal sampling variation. Choosing the model by its test score would use the test set for selection and make the reported test result optimistic, so the pre-defined rule was kept.

Placement readiness. Logistic Regression was clearly best (CV macro-F1 0.781 versus 0.710 for the next model). This is consistent with how the data was generated: the synthetic placement score is essentially a weighted sum of the inputs, which a linear model represents exactly, whereas tree ensembles must approximate it with many axis-aligned splits.

Placement probability. The logistic transformation makes this target non-linear, and the boosted-tree models performed best; XGBoost had the lowest CV RMSE (10.35). Ridge Regression, which cannot represent the S-shaped curve, had the highest error of the ensemble-free models except the single tree.

9.5 Confusion Matrices

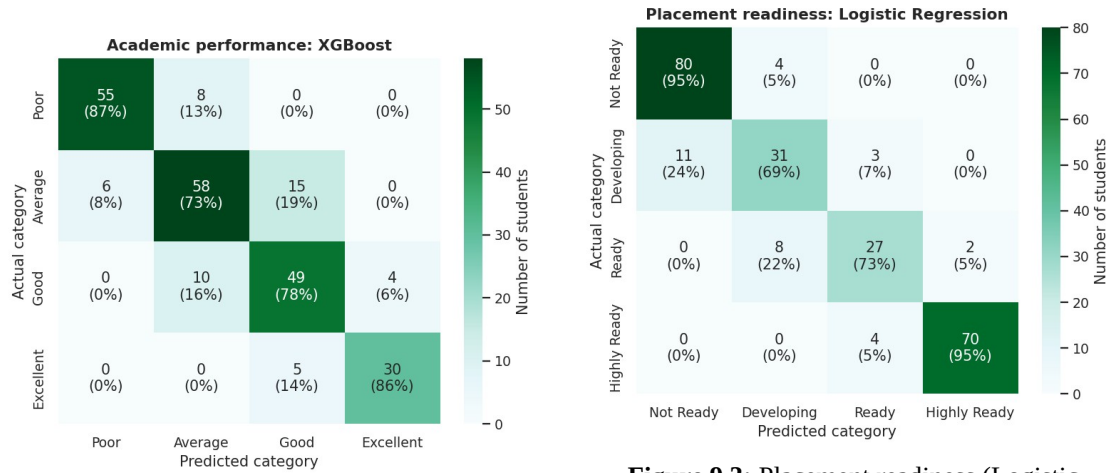


Figure 9.2: Academic performance (XGBoost)

Figure 9.3: Placement readiness (Logistic Regression)

Test-set confusion matrices. Rows are actual classes, columns predicted classes, percentages row-normalised. Every error falls in a neighbouring category — no Poor student was predicted Good or Excellent and vice versa. Not Ready (95 %) and Highly Ready (95 %) are recognised best; most readiness errors are Developing students predicted as Not Ready (11 of 45) and Ready students predicted as Developing (8 of 37). Confusion matrices for all ten classifiers are in Appendix C.

9.6 Per-class Results

Table 9.5: Per-class test-set metrics of the selected classifiers

Model	Class	Precision	Recall	F1	Support
Academic (XGBoost)	Poor	0.902	0.873	0.887	63
Academic (XGBoost)	Average	0.763	0.734	0.748	79
Academic (XGBoost)	Good	0.710	0.778	0.742	63
Academic (XGBoost)	Excellent	0.882	0.857	0.870	35
Readiness (Logistic Regression)	Not Ready	0.879	0.952	0.914	84
Readiness (Logistic Regression)	Developing	0.721	0.689	0.705	45
Readiness (Logistic Regression)	Ready	0.794	0.730	0.761	37
Readiness (Logistic Regression)	Highly Ready	0.972	0.946	0.959	74

The middle categories (Average/Good and Developing/Ready) are the hardest to predict, as expected: they are narrow bands of a continuous score and are surrounded by neighbours on both sides.

9.7 ROC Curves

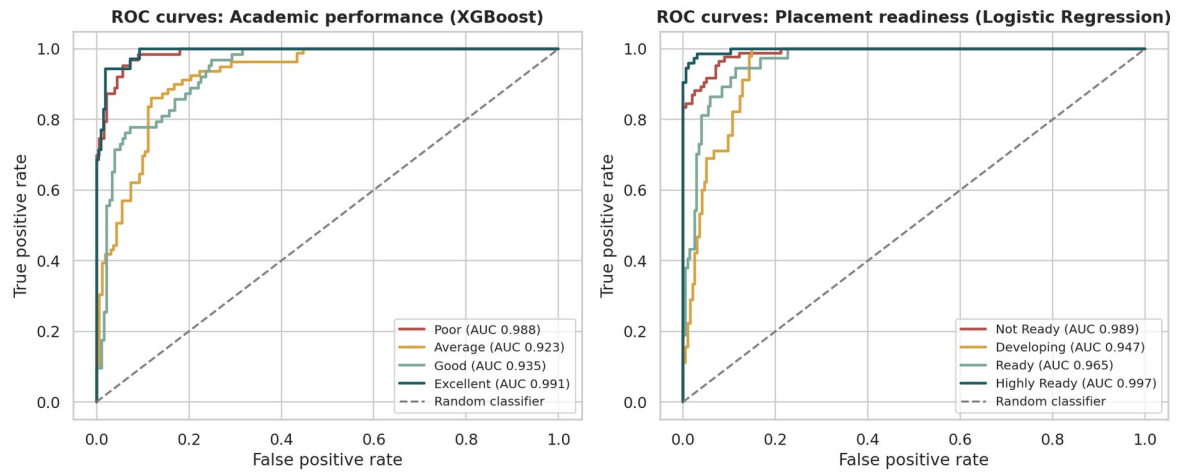


Figure 9.4: One-vs-rest ROC curves of the selected classifiers (test set)

Computed for this report from the saved models on the same 240-record test split. Extreme classes have AUCs close to 1; the middle classes have lower but still high AUCs.

9.8 Regression Diagnostics

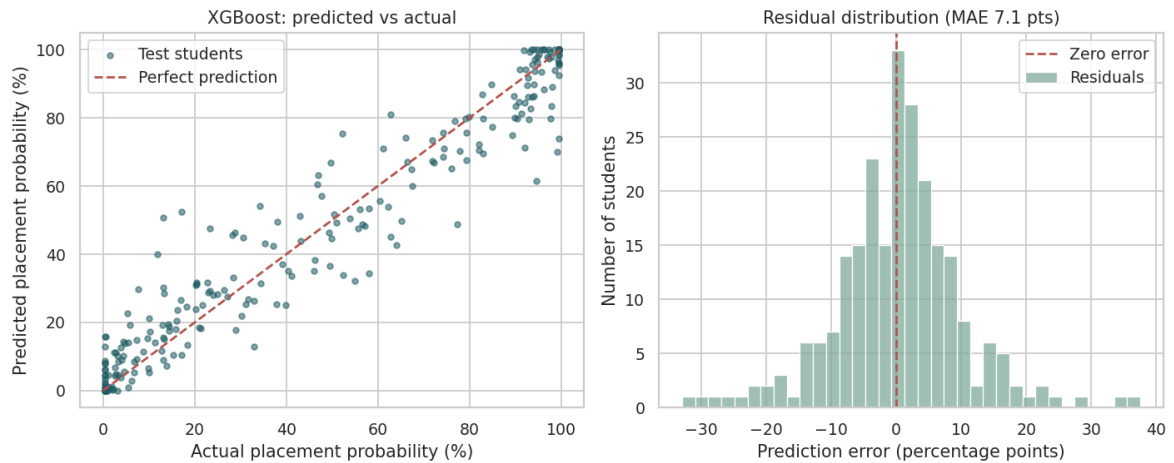


Figure 9.5: Predicted vs actual placement probability and residual distribution (XGBoost, test set)

Predictions follow the diagonal closely; residuals are centred on zero, with the largest errors in the middle of the range, where the S-shaped relationship is steepest.

9.9 Fairness Audit

Table 9.6: Outcomes by gender on the test set (gender is not a model input)

Gender	Test students	Readiness accuracy	Mean actual prob. (%)	Mean predicted prob. (%)	Predicted Ready or above
Female	87	88.5 %	50.67	50.64	46.0 %
Male	147	85.0 %	43.95	44.38	41.5 %
Other	6	100.0 %	73.60	71.56	83.3 %

Mean predicted probabilities closely track the actual values for each group. Differences between groups mirror differences in the (synthetic) data rather than the use of gender by the model. The 'Other' group contains only 6 test students, so its figures are not reliable.

9.10 Interpretation

The selected models perform well on this dataset: test macro-F1 of 0.812 (academic) and 0.835 (readiness), and R^2 of 0.930 for the probability. These numbers show that the pipeline recovers the relationships built into the synthetic generator. **They do not show that the models would perform similarly on real students**, whose outcomes depend on factors that are not in the data and whose feature–outcome relationships are unknown.

CHAPTER 10

EXPLAINABLE AI

10.1 Why Interpretability Matters in Education

- **Actionability.** A student told that their estimate is low needs to know what would help; a contribution chart points to specific levers such as internships or coding practice.
- **Trust and accountability.** Mentors can check whether a prediction rests on sensible factors before discussing it with a student.
- **Error detection.** Explanations can reveal when a model relies on an inappropriate or spurious feature.
- **Fairness.** Explanations make it possible to verify that demographic attributes play no role.

10.2 Global Feature Importance

Permutation importance [17] measures how much a model's test-set score (macro-F1 for classifiers, RMSE for the regressor) worsens when one feature's values are randomly shuffled, breaking its link with the target. Shares were normalised to sum to 100 % and grouped into impact levels: High (≥ 15 %), Medium-high (8–15 %), Medium (4–8 %) and Low (< 4 %).

Table 10.1: Top nine features by permutation importance for each selected model

Academic	Share	Impact	Readiness	Share	Impact	Probability	Share	Impact
CGPA	83.1 %	High	Coding / DSA score	19.6 %	High	Coding / DSA score	20.1 %	High
Attendance %	8.5 %	Medium-high	Internship	18.3 %	High	Internship	17.5 %	High
Study hours / week	4.7 %	Medium	CGPA	14.9 %	Medium-high	CGPA	16.8 %	High
Backlogs	1.8 %	Low	Projects	14.1 %	Medium-high	Projects	16.8 %	High
Projects	0.7 %	Low	Aptitude score	7.1 %	Medium	Technical skills score	10.2 %	Medium-high
Coding / DSA score	0.7 %	Low	Backlogs	6.3 %	Medium	Aptitude score	5.8 %	Medium
Communication score	0.4 %	Low	Certifications	5.5 %	Medium	Backlogs	3.7 %	Low
Aptitude score	0.1 %	Low	Technical skills score	5.4 %	Medium	Certifications	2.9 %	Low
Extracurricular score	0.0 %	Low	Communication score	4.3 %	Medium	Communication score	2.8 %	Low

For both placement models the four most important features are coding score, internship, CGPA and projects, together accounting for 66.8 % (readiness) and 71.2 % (probability) of total importance. Academic performance depends almost entirely on CGPA (83.1 %).

These rankings match the weights in the synthetic generator, which is a useful sanity check. Importance describes what the model relies on; it does not establish that a feature *causes* placement.

10.3 SHAP Explanations

SHAP [14] explains an individual prediction $f(x)$ as the sum of a base value (the average prediction over reference data) and one contribution per feature: $f(x) = \text{base} + \sum \phi_i$. For the probability model the contributions are in percentage points, so they can be read directly: "this feature moved the estimate up by 7.7 points". The project uses `shap.TreeExplainer` for the XGBoost models and `shap.LinearExplainer` for the Logistic Regression model, with 100 training records as the background sample.

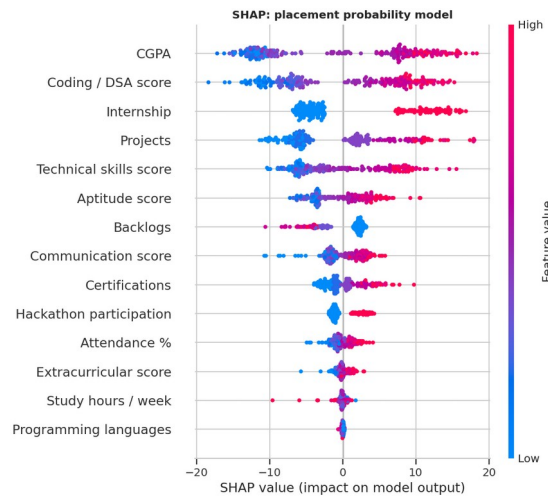


Figure 10.1: SHAP summary: placement probability model

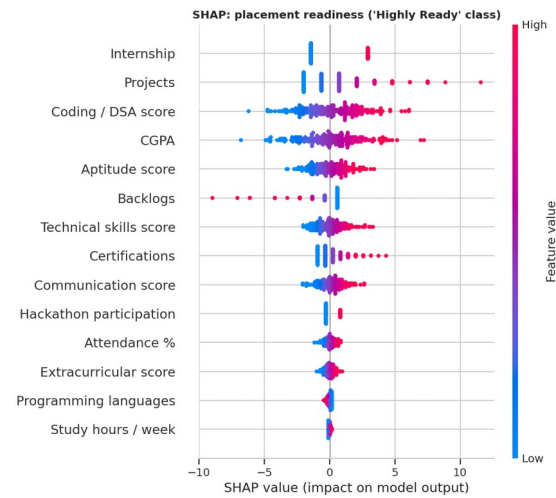


Figure 10.2: SHAP summary: readiness model ('Highly Ready' class)

Each dot is one test student; the x-position is the feature's contribution and the colour is the feature value (red high, blue low). High coding scores, CGPA, projects and technical skills push the estimate up, and having no internship pushes it down. For the linear readiness model, contributions increase monotonically with each feature value, reflecting its linear form.

10.4 Example Prediction Explanation

The "average student" sample profile (CGPA 7.1, attendance 79 %, no backlogs, coding 58, aptitude 60, communication 63, 2 projects, 1 certification, no internship, no hackathon, technical skills 47) receives: academic performance **Average** (confidence 51 %), placement readiness **Developing** (66 %) and estimated placement probability **45 %**, against a cohort-average base value of 48 %.

Table 10.2: SHAP contributions for the average-student example (top eight)

Feature	Student's value	Contribution (percentage points)	Direction
CGPA	7.1	+7.74	raises estimate
Internship	No	-6.22	lowers estimate
Technical skills score	47	-5.13	lowers estimate
Backlogs	0	+2.33	raises estimate

Feature	Student's value	Contribution (percentage points)	Direction
Projects	2	+2.12	raises estimate
Communication score	63	-2.08	lowers estimate
Coding / DSA score	58	+2.08	raises estimate
Hackathon participation	No	-1.96	lowers estimate

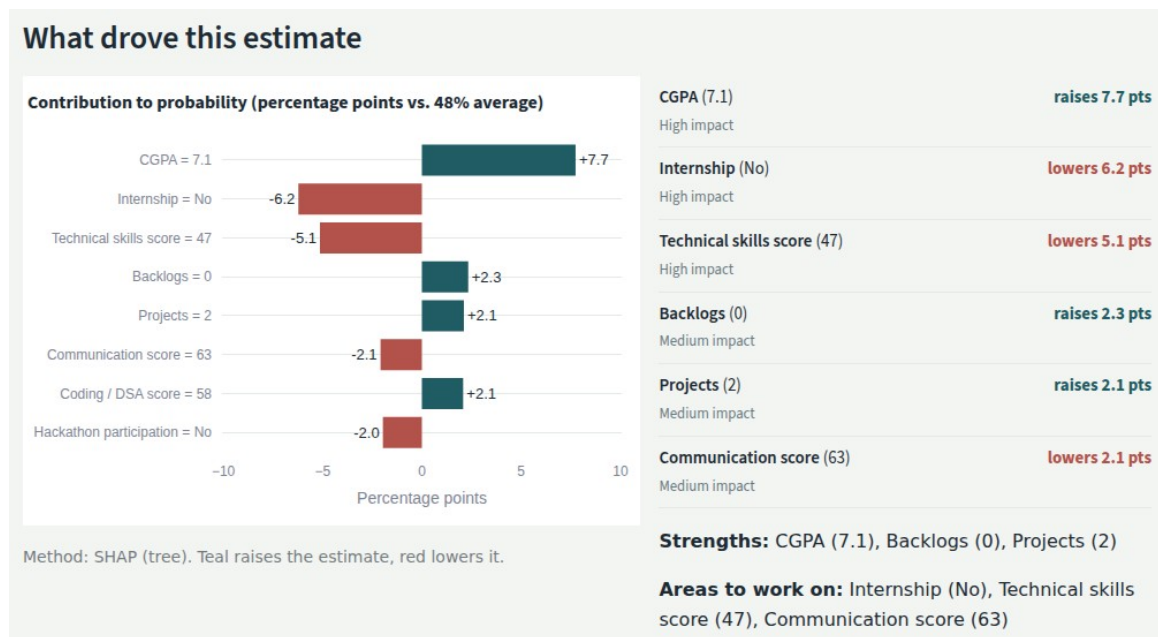


Figure 10.3: Explanation panel in the application for the same student

The app shows the same contributions as a chart and a list, adds an impact level for each, and summarises strengths and areas to work on.

The explanation is directly useful for mentoring: this student's above-average CGPA helps, but the absence of an internship and a below-average technical-skills score pull the estimate down. Table 14.1 (TC07) confirms that, with all other inputs unchanged, adding an internship raises this student's estimate from 45 % to 70 %.

CHAPTER 11

SYSTEM DESIGN

11.1 System Architecture Diagram

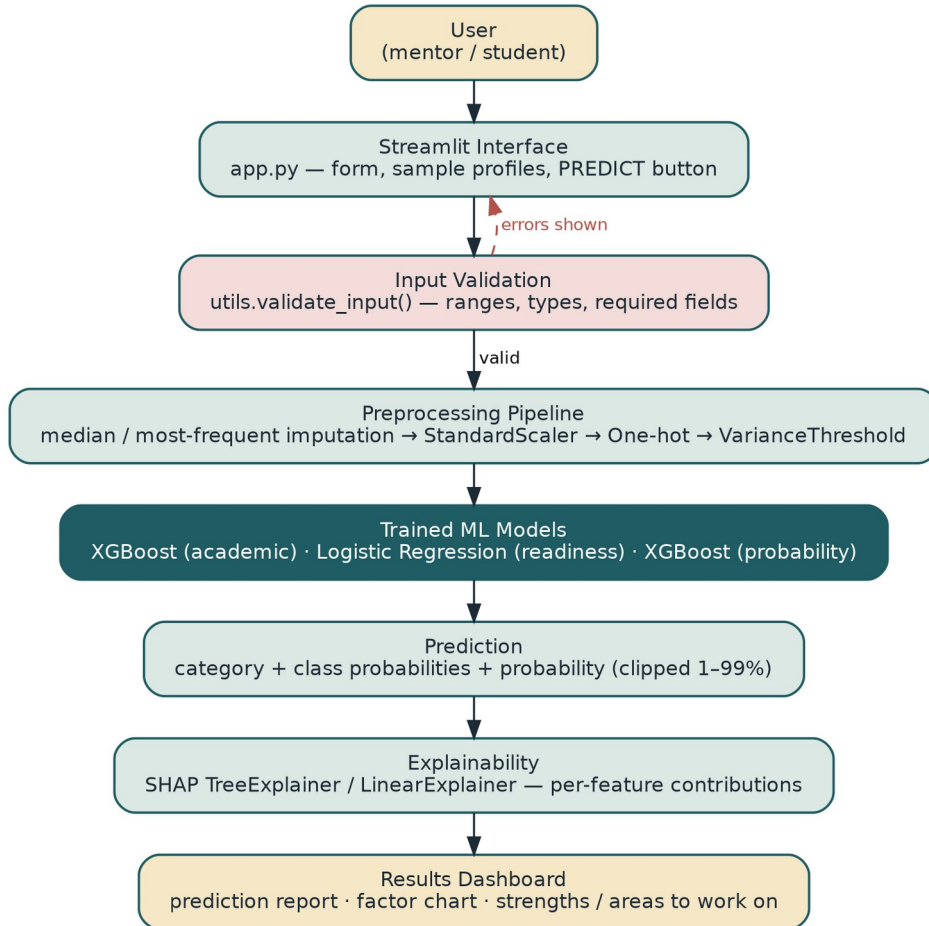


Figure 11.1: System architecture: request flow from user to results

Invalid input is returned to the interface with error messages and never reaches the models.

11.2 System Architecture (Component View)

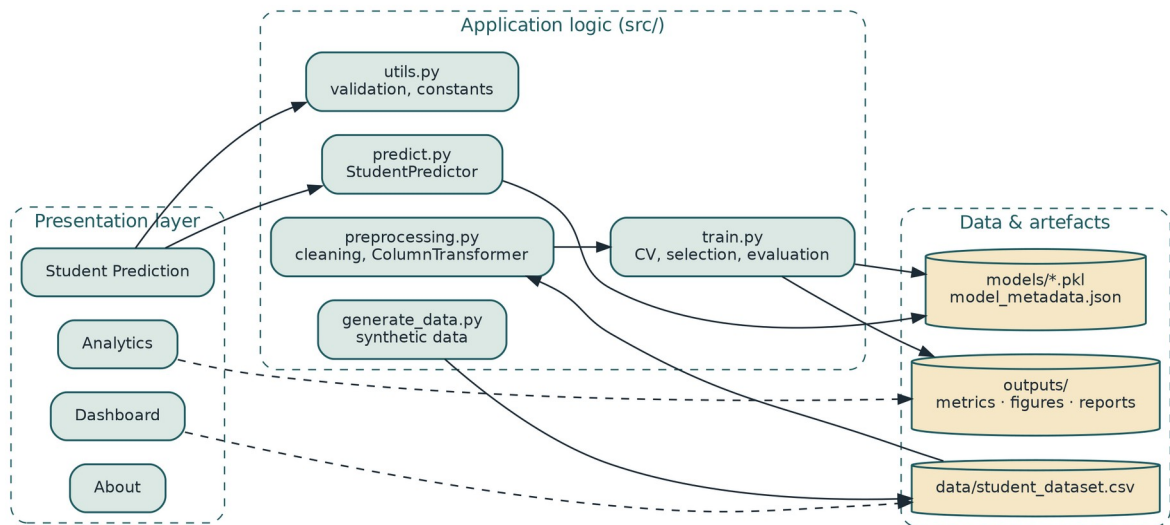


Figure 11.2: Component architecture: presentation layer, application logic and stored artefacts

The Streamlit pages call the validation and prediction modules; the training script produces the model files and outputs that the app reads.

11.3 Data-flow Diagram

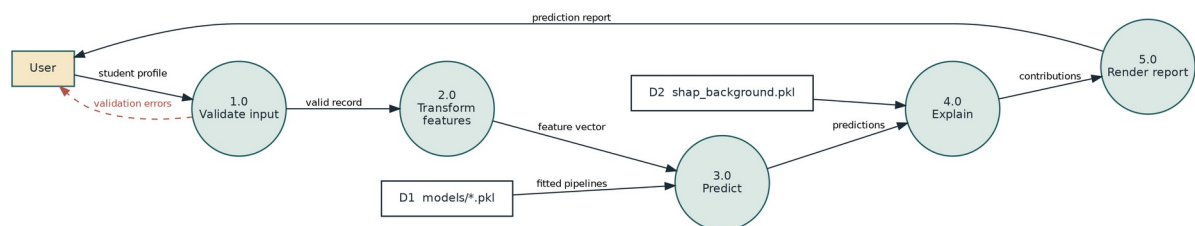


Figure 11.3: Level-1 data-flow diagram of the prediction process

Processes 1.0–5.0 transform the student profile into a prediction report; D1 and D2 are the stored model and SHAP background files.

11.4 ML Pipeline Diagram

The machine-learning pipeline used for training and evaluation is shown in Figure 6.1 (Chapter 6).

11.5 Application Workflow

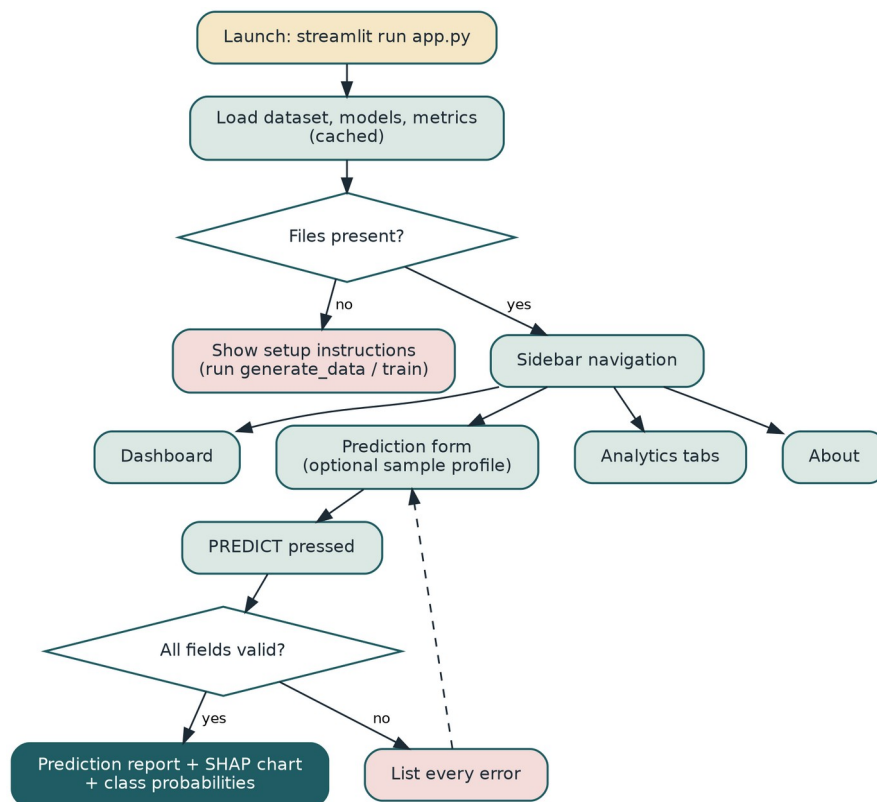


Figure 11.4: Application workflow

At start-up the app loads cached data and models; if files are missing it shows setup instructions instead of failing.

CHAPTER 12

IMPLEMENTATION

12.1 Project Structure

Table 12.1: Project modules and responsibilities

Path	Responsibility
app.py	Streamlit application: four pages, form, validation messages, prediction report, charts
src/generate_data.py	Synthetic data generator with injected data-quality issues
src/preprocessing.py	Loading, rule-based cleaning, IQR outlier report, preprocessing Pipeline
src/train.py	Cross-validated comparison, model selection, evaluation, figures, fairness audit, export
src/predict.py	StudentPredictor: validated, explained predictions
src/utls.py	Paths, feature lists, valid ranges, colours, validate_input()
notebooks/	01_data_generation, 02_eda, 03_model_training (executed)
models/	academic_model.pkl, placement_model.pkl, probability_model.pkl, preprocessing_pipeline.pkl, shap_background.pkl, model_metadata.json
outputs/	figures/, metrics/, reports/
tests/	test_cases.py (12 cases), test_app.py (3 Streamlit smoke tests)

12.2 Dataset Loading

`load_raw()` reads the CSV with pandas; `clean_dataset()` applies the rules of Section 6.1 and returns the cleaned frame and a report dictionary that is saved as JSON.

12.3 Preprocessing

`build_preprocessor()` (Listing 6.1) returns an unfitted Pipeline. It is combined with each estimator by `make_pipeline()` so that preprocessing is always fitted together with the model.

12.4 Model Training

For each task, `run_classification()` or `run_regression()` loops over the candidate models, cross-validates the full pipeline, fits it on the training set, evaluates it on the test set and records all metrics. The candidates are ranked by the CV criterion and the top one is selected.

```

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = {"accuracy": "accuracy", "precision": "precision_macro",
           "recall": "recall_macro", "f1": "f1_macro", "roc_auc": "roc_auc_ovr"}
for name, est in classifier_candidates().items():
    pipe = make_pipeline(clone(est))          # preprocess -> model
    cvr = cross_validate(pipe, X_train, ytr, cv=cv, scoring=scoring)
    pipe.fit(X_train, ytr)
    ...
table = pd.DataFrame(rows).sort_values("cv_f1", ascending=False)
best = table.loc[0, "model"]                 # selection by CV only

```

Listing 12.1: Cross-validated model comparison (src/train.py, abridged)

12.5 Model Saving

The selected pipelines are saved with `joblib.dump()`. Because each file contains the fitted preprocessing as well as the model, the application needs no separate preprocessing code. `model_metadata.json` records the selected models, selection rule, feature lists, split sizes and test metrics, and is read by the app's Dashboard and About pages.

12.6 Prediction

```

class StudentPredictor:
    def predict(self, record: dict, explain: bool = True) -> PredictionResult:
        errors = validate_input(record)
        if errors:
            raise InvalidInputError(errors)
        X = self.to_frame(record)
        acad_p = self.academic.predict_proba(X)[0]
        ready_p = self.placement.predict_proba(X)[0]
        prob = float(np.clip(self.probability.predict(X)[0], 1, 99))
        ... # categories = argmax; SHAP contributions if explain=True

```

Listing 12.2: Prediction entry point (src/predict.py, abridged)

12.7 Streamlit Interface

`app.py` loads the dataset, predictor and metrics once (`st.cache_data / st.cache_resource`) and routes between four pages with a sidebar control. The prediction page is an `st.form` in three columns with a *Start from* selector for sample profiles. Numeric widgets are deliberately unbounded so that out-of-range values reach `validate_input()` and receive an explicit error message, as the specification requires.

12.8 Visualisation

Static figures are produced with Matplotlib and Seaborn (130 dpi); the app uses Plotly. Each category keeps the same colour everywhere, e.g. Poor/Not Ready in red and Excellent/Highly Ready in dark teal.

12.9 Explainability

`StudentPredictor` caches one SHAP explainer per model (`TreeExplainer`, else `LinearExplainer`) and, for multi-class models, uses the contributions for the predicted class. If SHAP is unavailable, an occlusion fallback keeps the app working.

CHAPTER 13

APPLICATION SCREENSHOTS

All screenshots in this chapter were captured from the running application (`streamlit run app.py`) with a headless Chromium browser at a width of 1,440 pixels. Figures 13.4–13.6 are crops of Figure 13.3.

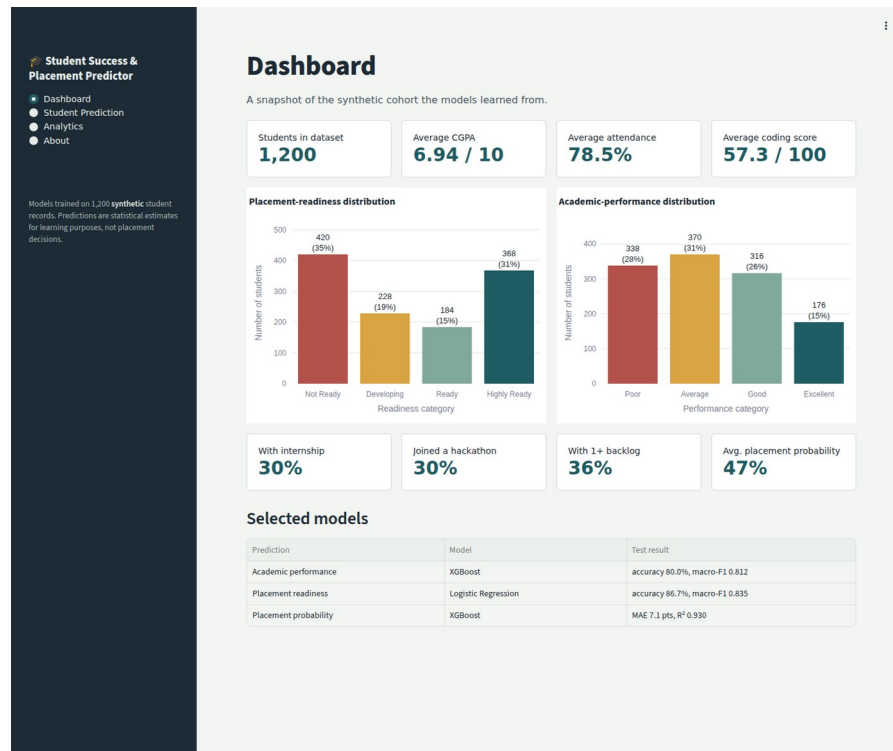


Figure 13.1: Dashboard

Key dataset metrics (1,200 students, average CGPA 6.94, average attendance 78.5 %, average coding score 57.3), the readiness and academic-performance distributions, and the selected models with their test results.

The screenshot shows a web application titled "Student Success & Placement Predictor". The left sidebar contains a navigation menu with "Dashboard", "Student Prediction" (selected), "Analytics", and "About". Below the menu is a disclaimer: "Models trained on 1,200 synthetic student records. Predictions are statistical estimates for learning purposes, not placement decisions." The main content area is titled "Student Prediction" and includes a brief instruction: "Enter a student's academic and skills profile, then press PREDICT. Every field is checked before the models run." Below this is a "Start from" dropdown menu set to "Blank form (typical values)". The form itself is divided into three columns: "Academics", "Skills", and "Experience & profile". Each column contains several input fields with numerical values and minus/plus adjustment buttons. The "Academics" column includes CGPA (7.00), Attendance % (80.0), Backlogs (0), Study hours per week (14.0), and Aptitude score (59). The "Skills" column includes Coding / DSA score (57), Technical skills score (46), Communication score (64), Programming languages known (2), and Extracurricular score (58). The "Experience & profile" column includes Projects completed (2), Certifications (1), Internship experience (radio buttons for Yes/No, with No selected), Hackathon participation (radio buttons for Yes/No, with No selected), Age (21), and Gender (Male). A "PREDICT" button is located at the bottom left of the form area.

Academics	Skills	Experience & profile
CGPA (0-10) 7.00 - +	Coding / DSA score (0-100) 57 - +	Projects completed 2 - +
Attendance % (0-100) 80.0 - +	Technical skills score (0-100) 46 - +	Certifications 1 - +
Backlogs 0 - +	Communication score (0-100) 64 - +	Internship experience ☐ Yes ☒ No
Study hours per week 14.0 - +	Programming languages known 2 - +	Hackathon participation ☐ Yes ☒ No
Aptitude score (0-100) 59 - +	Extracurricular score (0-100) 58 - +	Age 21 - +
		Gender Male

PREDICT

Figure 13.2: Student input form

Sixteen inputs grouped into Academics, Skills and Experience & profile, the sample-profile selector and the PREDICT button. The help icon on Gender states that gender and age are not model inputs.

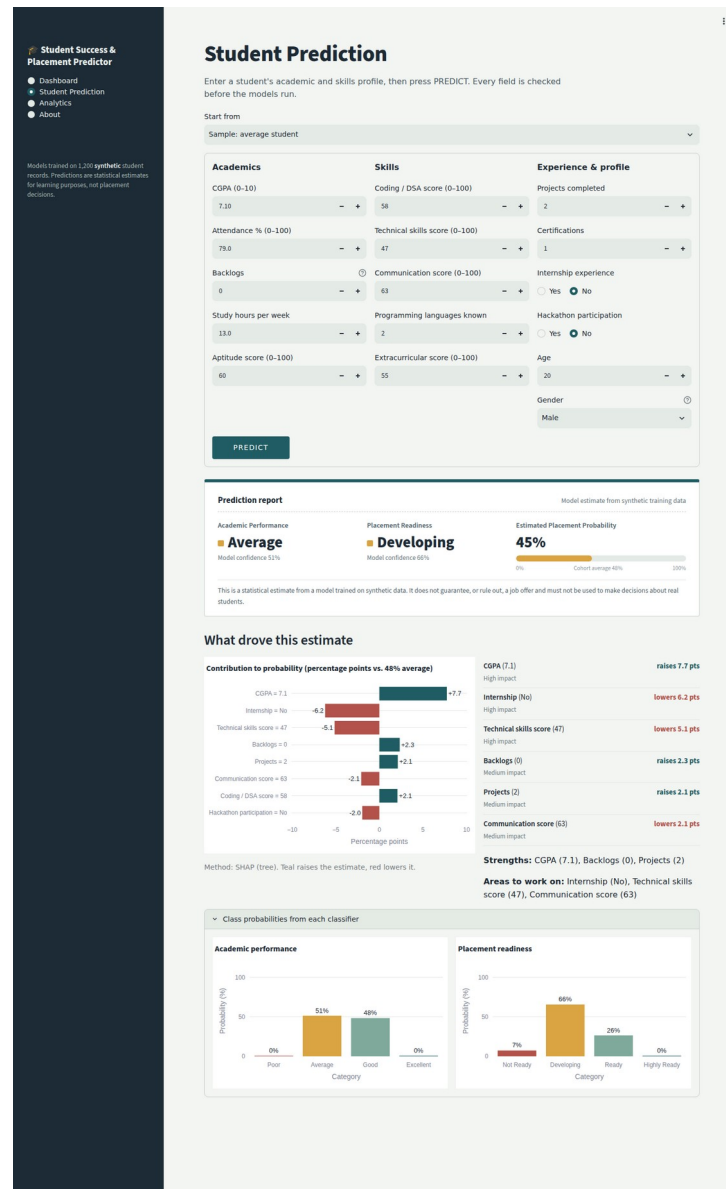


Figure 13.3: Prediction screen for the average-student sample

After PREDICT, the prediction report, the explanation panel and the expandable class-probability charts appear below the form.

Academic Performance
Average
Model confidence 51%

Figure 13.4: Academic performance result

Placement Readiness
Developing
Model confidence 66%

Figure 13.5: Placement readiness result

Estimated Placement Probability
45%
Cohort average 48%

Figure 13.6: Estimated placement probability

Each category is shown with its colour code and the model's confidence (probability of the predicted class). The probability bar shows the estimate on a 0–100 % scale with the cohort average as reference.

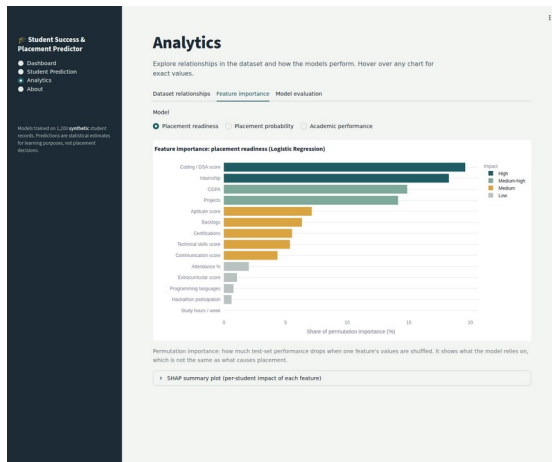


Figure 13.7: Feature importance in the Analytics page

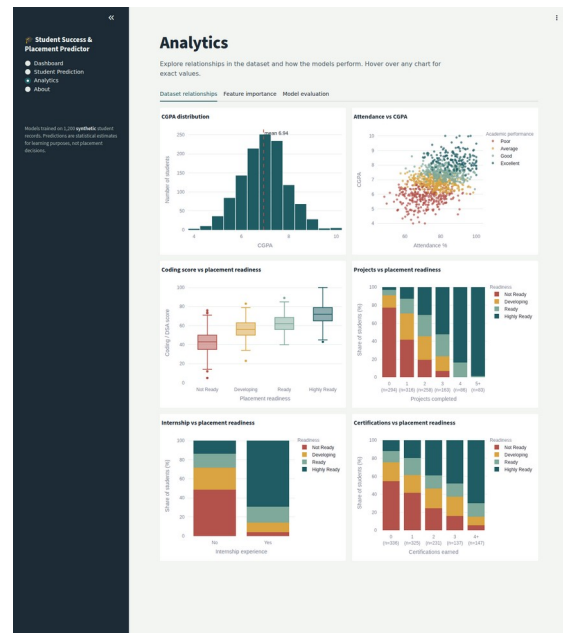


Figure 13.8: Analytics: dataset relationships

The Analytics page has three tabs. Feature importance lets the user switch between the three models, with bars coloured by impact level. Dataset relationships holds interactive charts of CGPA distribution, attendance vs CGPA, and coding score, projects, internship and certifications vs placement readiness.

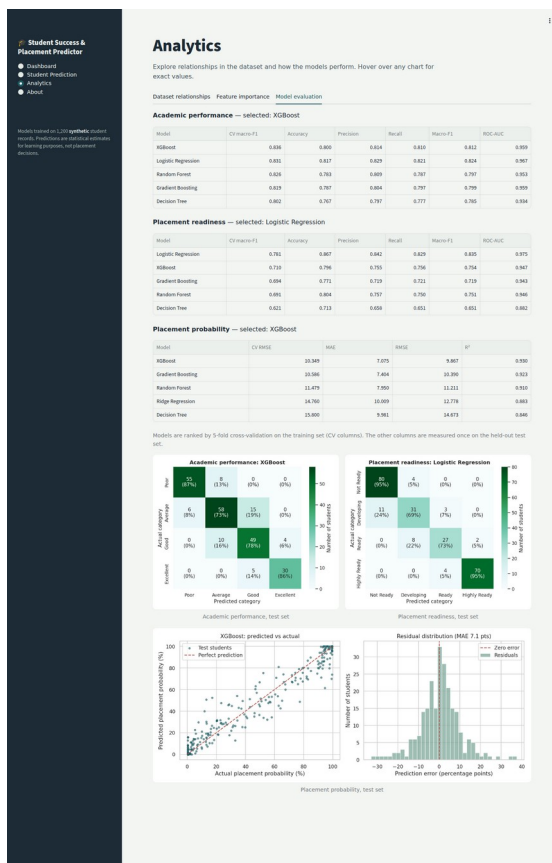


Figure 13.9: Analytics: model evaluation tab

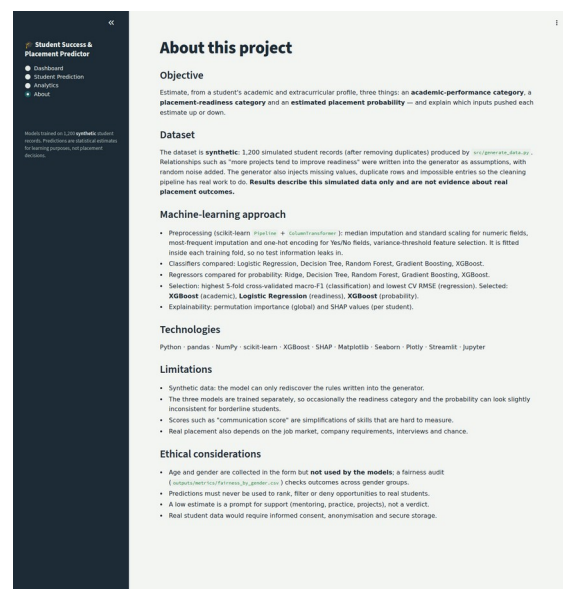


Figure 13.10: About section

The model-evaluation tab reproduces the comparison tables, confusion matrices and regression diagnostics inside the application. The About section states the objective, dataset, ML approach, technologies, limitations and ethical considerations.

CHAPTER 14

TESTING

14.1 Test Approach

Testing was carried out at three levels: (1) **automated prediction tests** (tests/test_cases.py, 12 cases run with pytest against the saved models); (2) **additional boundary, missing-input and type tests** (6 cases, run against the same StudentPredictor for this report); and (3) **application smoke tests** (tests/test_app.py) using Streamlit's AppTest, which render every page, press PREDICT and submit invalid values. In addition, the running application was exercised in a browser to capture the screenshots in Chapter 13.

14.2 Test Cases

Table 14.1: Test cases with expected and actual results

Test ID	Test Case	Input	Expected Result	Actual Result	Status
TC01	High-performing student	CGPA 9.1, att. 93, coding 86, 4 projects, internship Yes, hackathon Yes	Academic Good/Excellent; readiness Ready/Highly Ready; probability $\geq 70\%$	Excellent / Highly Ready / 99%	PASS
TC02	Average-performing student	CGPA 7.1, att. 79, coding 58, 2 projects, no internship	Academic Average/Good; readiness Developing/Ready; probability 20–70%	Average / Developing / 45%	PASS
TC03	Low-performing student	CGPA 5.4, att. 58, 4 backlogs, coding 34, 0 projects	Academic Poor; readiness Not Ready; probability $< 20\%$	Poor / Not Ready / 1%	PASS
TC04	Student with multiple backlogs	Average profile, backlogs = 5	Academic category drops to Poor/Average; probability lower than the same student with 0 backlogs; backlogs is a negative factor	Average / Not Ready / 36%	PASS
TC05	Excellent coding score	Average profile, coding = 95	Probability higher than the average student; coding is among the top-3 positive factors	Average / Ready / 59%	PASS
TC06	Poor attendance	Average profile, attendance = 45	Academic category Poor/Average; attendance contributes negatively	Average / Developing / 40%	PASS
TC07	Student with internship	Average profile, internship = Yes	Probability higher than the identical student without internship; internship is a positive factor	Average / Ready / 70%	PASS
TC08	Student without internship	Average profile, internship = No	Valid prediction returned; internship contributes negatively (below-average experience)	Average / Developing / 45%	PASS

Test ID	Test Case	Input	Expected Result	Actual Result	Status
TC09	Invalid CGPA	Average profile, CGPA = 11.0	Rejected with message 'CGPA must be between 0 and 10'	Rejected: CGPA must be between 0 and 10 (you entered 11).	PASS
TC10	Invalid attendance	Average profile, attendance = 105	Rejected with message 'Attendance % must be between 0 and 100'	Rejected: Attendance % must be between 0 and 100 (you entered 105).	PASS
TC11	Negative backlogs and projects	backlogs = -1, projects = -2	Rejected with two 'cannot be negative' messages	Rejected: Backlogs cannot be negative (you entered -1). / Projects cannot be negative (you entered -2).	PASS
TC12	Gender changed, everything else identical	Average profile, gender = Female	Identical predictions (gender is not a model input)	Average / Developing / 45%	PASS
TC13	Boundary: minimum valid values	cgpa=0, attendance=0, all scores=0, study hours=0	Accepted; lowest categories	Poor / Not Ready / 1%	PASS
TC14	Boundary: maximum valid values	cgpa=10, attendance=100, all scores=100	Accepted; highest categories	Excellent / Highly Ready / 99%	PASS
TC15	Boundary: just above maximum (CGPA 10.01)	cgpa=10.01	Rejected with CGPA range message	Rejected: CGPA must be between 0 and 10 (you entered 10.01).	PASS
TC16	Missing input (CGPA left empty)	cgpa field absent	Rejected: 'CGPA is required.'	Rejected: CGPA is required.	PASS
TC17	Non-numeric input	coding_score='abc'	Rejected: must be a number	Rejected: Coding / DSA score must be a number.	PASS
TC18	Fractional count	projects=2.5	Rejected: must be a whole number	Rejected: Projects must be a whole number (you entered 2.5).	PASS

Results are shown as *Academic performance / Placement readiness / Probability*. All 18 cases passed. Cases TC01–TC12 are automated; TC13–TC18 were executed for this report with the same predictor. TC08 and TC02 use the same input by design (TC08 checks the internship contribution), and TC12 confirms that changing gender alone leaves every output unchanged.

14.3 Application Tests

Table 14.2: Streamlit application smoke tests (tests/test_app.py)

Test	Procedure	Result
Every page renders	AppTest: open Dashboard, Student Prediction, Analytics and About	No exceptions — PASS
PREDICT produces a report	Select the high-performer sample, press PREDICT	Prediction report rendered, no error — PASS
Invalid input is rejected	Enter CGPA 11.5 and attendance 120, press PREDICT	Both error messages shown, no report — PASS

Running `pytest -v tests/` executed 15 tests (12 prediction cases and 3 application tests); all passed.

The screenshot shows the 'Student Prediction' web application. On the left is a dark sidebar with the title 'Student Success & Placement Predictor' and navigation links: Dashboard, Student Prediction (active), Analytics, and About. Below the links is a note: 'Models trained on 1,200 synthetic student records. Predictions are statistical estimates for learning purposes, not placement decisions.' The main content area is titled 'Student Prediction' and includes instructions: 'Enter a student's academic and skills profile, then press PREDICT. Every field is checked before the models run.' Below this is a 'Start from' dropdown menu set to 'Sample: student at risk'. The form is divided into three columns: 'Academics', 'Skills', and 'Experience & profile'. Each column contains several input fields with numerical values or selected options. At the bottom of the form is a green 'PREDICT' button. Below the button, a red error message box states: 'Please fix the following before predicting:' followed by three bullet points: 'CGPA must be between 0 and 10 (you entered 11.5).', 'Attendance % must be between 0 and 100 (you entered 120).', and 'Backlogs cannot be negative (you entered -2).'

Academics	Skills	Experience & profile
CGPA (0-10) 11.50	Coding / DSA score (0-100) 34	Projects completed 0
Attendance % (0-100) 120.0	Technical skills score (0-100) 24	Certifications 0
Backlogs -2	Communication score (0-100) 52	Internship experience ☐ Yes ☒ No
Study hours per week 5.0	Programming languages known 1	Hackathon participation ☐ Yes ☒ No
Aptitude score (0-100) 41	Extracurricular score (0-100) 40	Age 22
		Gender Male

Please fix the following before predicting:

- CGPA must be between 0 and 10 (you entered 11.5).
- Attendance % must be between 0 and 100 (you entered 120).
- Backlogs cannot be negative (you entered -2).

Figure 14.1: Validation in the application

Entering CGPA 11.5, attendance 120 and backlogs -2 produces one clear message per field, and no prediction is made.

CHAPTER 15

RESULTS

15.1 Selected Models and Metrics

Table 15.1: Summary of final model results

Prediction	Selected model	Selection score (CV)	Test results
Academic performance	XGBoost	macro-F1 0.836	Accuracy 0.800, macro-F1 0.812, ROC-AUC 0.959
Placement readiness	Logistic Regression	macro-F1 0.781	Accuracy 0.867, macro-F1 0.835, ROC-AUC 0.975
Placement probability	XGBoost	RMSE 10.35	MAE 7.07, RMSE 9.87, R ² 0.930

15.2 Prediction Examples

Table 15.2: Predictions for the three sample profiles in the application

Sample profile	Academic	Readiness	Probability	Main factors (SHAP, points)
High performer (CGPA 9.1, coding 86, internship)	Excellent	Highly Ready	99 %	CGPA +10.4, Projects +9.1, Coding +9.0
Average student (CGPA 7.1, coding 58, no internship)	Average	Developing	45 %	CGPA +7.7, Internship -6.2, Technical skills -5.1
Student at risk (CGPA 5.4, 4 backlogs, coding 34)	Poor	Not Ready	1 %	CGPA -11.2, Coding -10.5, Technical skills -5.6

15.3 Application Performance

- Loading the three models and SHAP background took 2.422 s at start-up (cached afterwards).
- One explained prediction took 26.3 ms on average and 27.2 ms at the 95th percentile over 50 runs.
- Regenerating the dataset with seed 42 reproduced a byte-identical CSV file, and retraining selected the same three models.

15.4 Key Observations

- No single algorithm was best for every task: a linear model won where the underlying relationship was linear, boosted trees where it was not.
- All classification errors on the test set were between adjacent categories.
- Coding score, internship, CGPA and projects were the strongest placement drivers — exactly the relationships built into the generator.
- Explanations made the effect of each input concrete (e.g. an internship adds 25 points to the average student's estimate).

CHAPTER 16

LIMITATIONS

16.1 Dataset Limitations

The features are simplified indicators: a single communication score cannot represent interview performance, and there is no information about branch, college tier, company requirements or job-market conditions.

16.2 Synthetic-data Limitations

Because the data was generated from explicit formulas, a model can at best rediscover those formulas. High scores therefore measure how learnable the simulation is, not how predictable real placements are. Real relationships are likely to be weaker, noisier and different in shape.

16.3 Dataset Size

With 960 training and 240 test records, metric estimates carry noticeable uncertainty (e.g. ± 0.025 standard deviation in academic CV macro-F1), and small groups such as the six 'Other' test students give unreliable subgroup statistics.

16.4 Potential Bias

Age and gender are excluded from the models, but other features can act as proxies [19]: in real data, access to internships or hackathons can depend on socio-economic background, so a model that rewards them could disadvantage some groups. Real deployment would require a fairness audit on real data.

16.5 Model Generalisation

The models were validated only on data from the same generator, with no temporal validation (training on one batch, testing on a later one), which real use would require.

16.6 Placement Outcomes Depend on Many Factors

Real outcomes also depend on recruiting companies, economic conditions, interview performance, personal choices and chance — none of which is in the data.

16.7 Predictions Are Not Guarantees

The application states on every prediction that its output is a statistical estimate from a model trained on synthetic data and does not guarantee, or rule out, a job offer. It must not be used to rank, filter or deny opportunities to real students.

16.8 Need for Real-world Validation

Before any practical use, the pipeline would have to be retrained and validated on consented, anonymised institutional data, with calibration checks, subgroup analysis and review by the institution.

CHAPTER 17

FUTURE SCOPE

Table 17.1: Possible extensions

Extension	Description
Larger real-world datasets	Retrain and validate on several years of consented, anonymised institutional records.
University-specific datasets	Train separate or adapted models per institution or branch, since recruitment patterns differ.
Deep learning	Evaluate tabular neural networks once much larger datasets are available; with 1,200 records, tree ensembles and linear models are more appropriate.
NLP analysis of résumés	Extract skills, projects and experience from résumés automatically instead of relying on manual scores.
Personalised skill recommendations	Build a what-if planner that shows how the estimate changes if the student completes an internship or two more projects.
Job recommendation system	Match student profiles to job roles or companies based on required skills.
Integration with placement portals	Connect to college ERP or placement portals, with proper authentication and access control.
Real-time analytics	Update dashboards as new assessment results are entered.
Explainable AI improvements	Add counterfactual explanations, probability calibration and prediction intervals; use a single ordinal or multi-output model so that the three outputs are consistent by construction.
Hyperparameter tuning	Apply GridSearchCV or Bayesian optimisation within nested cross-validation.
Cloud deployment	Deploy on Streamlit Community Cloud or in a Docker container.
Mobile application	Provide a mobile front end for students to track their own progress.

CHAPTER 18

CONCLUSION

This project addressed the problem of estimating a student's academic performance and placement readiness early and transparently. A complete system was developed: a documented synthetic dataset of 1,200 students with realistic data-quality problems; a leakage-free preprocessing pipeline; a comparison of Logistic Regression, Decision Tree, Random Forest, Gradient Boosting and XGBoost for each of three tasks; SHAP-based explanations; a four-page Streamlit application with full input validation; and an automated test suite.

Using a pre-defined cross-validation rule, XGBoost was selected for academic performance (test macro-F1 0.812), Logistic Regression for placement readiness (test macro-F1 0.835, accuracy 0.867) and XGBoost for placement probability (test MAE 7.07 points, R^2 0.930). Coding score, internship experience, CGPA and projects were the most influential factors for placement, and every prediction in the application is accompanied by an explanation of these contributions. All 18 test cases and 3 application tests passed.

The main limitation is that the data is synthetic: the results demonstrate a correct and reproducible methodology rather than real-world predictive power, and the model cannot guarantee any student's placement. Within that limit, the project's academic contribution is a complete, explainable and honestly evaluated ML application that can be retrained on real, consented data in the future.

REFERENCES

- [1] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 40, no. 6, pp. 601–618, 2010.
- [2] R. S. J. d. Baker and K. Yacef, "The state of educational data mining in 2009: A review and future visions," *Journal of Educational Data Mining*, vol. 1, no. 1, pp. 3–17, 2009.
- [3] P. Cortez and A. Silva, "Using data mining to predict secondary school student performance," in *Proc. 5th Future Business Technology Conference (FUBUTEC 2008)*, Porto, Portugal, 2008, pp. 5–12.
- [4] S. Kotsiantis, C. Pierrakeas, and P. Pintelas, "Predicting students' performance in distance learning using machine learning techniques," *Applied Artificial Intelligence*, vol. 18, no. 5, pp. 411–426, 2004.
- [5] A. M. Shahiri, W. Husain, and N. A. Rashid, "A review on predicting student's performance using data mining techniques," *Procedia Computer Science*, vol. 72, pp. 414–422, 2015.
- [6] L. S. Maurya, M. S. Hussain, and S. Singh, "Developing classifiers through machine learning algorithms for student placement prediction based on academic performance," *Applied Artificial Intelligence*, vol. 35, no. 6, pp. 403–420, 2021, doi: 10.1080/08839514.2021.1901032.
- [7] M. Kumar, N. Walia, S. Bansal, G. Kumar, and K. Cengiz, "Predicting college students' placements based on academic performance using machine learning approaches," *International Journal of Modern Education and Computer Science*, vol. 15, no. 6, 2023.
- [8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [11] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.
- [12] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ: Wiley, 2013.
- [13] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York: Springer, 2009.
- [14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.
- [15] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

- [16] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, Art. 15, 2012.
- [17] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [18] C. Molnar, *Interpretable Machine Learning*, 2nd ed., 2022. [Online]. Available: <https://christophm.github.io/interpretable-ml-book/>
- [19] S. Barocas, M. Hardt, and A. Narayanan, *Fairness and Machine Learning: Limitations and Opportunities*. Cambridge, MA: MIT Press, 2023. [Online]. Available: <https://fairmlbook.org>
- [20] scikit-learn developers, "scikit-learn User Guide." [Online]. Available: https://scikit-learn.org/stable/user_guide.html
- [21] Python Software Foundation, "Python 3 Documentation." [Online]. Available: <https://docs.python.org/3/>
- [22] Streamlit Inc., "Streamlit Documentation." [Online]. Available: <https://docs.streamlit.io>
- [23] The pandas development team, "pandas documentation." [Online]. Available: <https://pandas.pydata.org/docs/>
- [24] XGBoost developers, "XGBoost Documentation." [Online]. Available: <https://xgboost.readthedocs.io>
- [25] SHAP contributors, "SHAP documentation." [Online]. Available: <https://shap.readthedocs.io>
- [26] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
- [27] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [28] M. L. Waskom, "seaborn: statistical data visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021.
- [29] Plotly Technologies Inc., "Plotly Python Open Source Graphing Library." [Online]. Available: <https://plotly.com/python/>
- [30] T. Kluyver et al., "Jupyter Notebooks – a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, IOS Press, 2016, pp. 87–90.
- [31] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference*, 2010, pp. 56–61.
- [32] Joblib developers, "Joblib: running Python functions as pipeline jobs." [Online]. Available: <https://joblib.readthedocs.io>

APPENDIX A — DATASET SAMPLE

Table A.1 shows the first six records of `data/student_dataset.csv` exactly as stored (before cleaning). 'nan' denotes an injected missing value.

Table A.1: Raw dataset sample, columns 1–10

ID	age	gender	cgpa	Att. %	backlogs	Coding	Aptitude	Comm.	projects
STU00740	20	Female	4.87	72.0	3	30.0	34	55.0	0
STU00789	20	Male	5.82	80.3	1	43.0	48	48.0	0
STU00044	23	Male	7.56	75.8	0	68.0	66	73.0	1
STU00156	20	Male	9.23	90.1	0	85.0	70	61.0	5
STU00495	20	Male	6.35	75.4	2	51.0	50	66.0	1
STU00971	22	Male	5.86	68.1	1	35.0	58	63.0	0

Table A.2: Raw dataset sample, columns 11–20

ID	Certs	Intern.	Hack.	Extra-curr.	Tech. skills	Langs	Study h/wk	Academic	Readiness	Prob. %
STU00740	0.0	No	No	79.0	22	1	7.7	Poor	Not Ready	2.1
STU00789	nan	No	No	62.0	28	4	12.3	Poor	Not Ready	5.6
STU00044	0.0	No	Yes	53.0	44	3	13.9	Good	Developing	47.7
STU00156	3.0	Yes	Yes	54.0	73	3	22.9	Excellent	Highly Ready	99.5
STU00495	0.0	No	No	59.0	32	1	7.3	Poor	Not Ready	7.7
STU00971	0.0	No	No	48.0	16	2	11.7	Poor	Not Ready	1.8

APPENDIX B — IMPORTANT CODE

B.1 Input validation (src/utils.py, abridged)

```
def validate_input(record: dict) -> list[str]:
    errors = []
    for col, (lo, hi, is_int) in VALID_RANGES.items():
        label = FEATURE_LABELS.get(col, col)
        if col not in record or record[col] is None or record[col] == "":
            errors.append(f"{label} is required."); continue
        try:
            value = float(record[col])
        except (TypeError, ValueError):
            errors.append(f"{label} must be a number."); continue
        if value < lo:
            errors.append(f"{label} cannot be negative (you entered {value:g}).")
    if lo == 0
        else f"{label} must be at least {lo:g} (you entered {value:g}).")
    elif value > hi:
        errors.append(f"{label} must be between {lo:g} and {hi:g} (you entered {value:g}).")
    elif is_int and not float(value).is_integer():
        errors.append(f"{label} must be a whole number (you entered {value:g}).")
    ... # Yes/No and gender checks
    return errors
```

B.2 Rule-based cleaning (src/preprocessing.py, abridged)

```
def clean_dataset(df):
    df = df.drop_duplicates()
    df = df.dropna(subset=[TARGET_ACADEMIC, TARGET_READINESS, TARGET_PROBABILITY])
    for col, (lo, hi, _) in VALID_RANGES.items():
        mask = (df[col] < lo) | (df[col] > hi)
        df.loc[mask, col] = np.nan # imputed later, inside the pipeline
    return df.reset_index(drop=True), report
```

B.3 Per-prediction SHAP contributions (src/predict.py, abridged)

```
def _contributions(self, name, pipe, X, class_index=None):
    explainer, method, pre, est, bg = self._explainer(name, pipe) # Tree or Linear
    Xt = pre.transform(X)
    sv = explainer(Xt)
    values, base = np.asarray(sv.values), np.asarray(sv.base_values)
    if values.ndim == 3: # multi-class: take predicted class
        values = values[:, :, class_index]
    return values[0], float(np.ravel(base)[0]), transformed_feature_names(pre), method
```

B.4 Streamlit prediction flow (app.py, abridged)

```
with st.form("student_form"):
    ...                                # 16 inputs in three columns
    submitted = st.form_submit_button("PREDICT")
if submitted:
    record = {k[3:]: v for k, v in st.session_state.items() if k.startswith("in_")}
    errors = validate_input(record)
    if errors:
        st.error("Please fix the following before predicting:\n" + "\n".join(f"-{e}" for e in errors))
    else:
        render_result(predictor.predict(record))
```

APPENDIX C — MODEL METRICS

Complete metric tables from outputs/metrics/. CV values are means over five folds; test values are from the 240-record test set.

Table C.1: All metrics: academic performance

Model	CV Acc.	CV Prec.	CV Rec.	CV F1	CV AUC	Test Acc.	Test Prec.	Test Rec.	Test F1	Test AUC
XGBoost	0.834	0.843	0.833	0.836	0.963	0.800	0.814	0.810	0.812	0.959
Logistic Regression	0.833	0.844	0.825	0.831	0.969	0.817	0.829	0.821	0.824	0.967
Random Forest	0.826	0.845	0.818	0.826	0.957	0.783	0.809	0.787	0.797	0.953
Gradient Boosting	0.817	0.827	0.816	0.819	0.957	0.787	0.804	0.797	0.799	0.959
Decision Tree	0.801	0.818	0.799	0.802	0.937	0.767	0.797	0.777	0.785	0.934

Table C.2: All metrics: placement readiness

Model	CV Acc.	CV Prec.	CV Rec.	CV F1	CV AUC	Test Acc.	Test Prec.	Test Rec.	Test F1	Test AUC
Logistic Regression	0.823	0.788	0.781	0.781	0.963	0.867	0.842	0.829	0.835	0.975
XGBoost	0.765	0.716	0.710	0.710	0.936	0.796	0.755	0.756	0.754	0.947
Gradient Boosting	0.754	0.702	0.693	0.694	0.926	0.771	0.719	0.721	0.719	0.943
Random Forest	0.760	0.706	0.690	0.691	0.934	0.804	0.757	0.750	0.751	0.946
Decision Tree	0.692	0.629	0.624	0.621	0.862	0.713	0.658	0.651	0.651	0.882

Table C.3: All metrics: placement probability

Model	CV MAE	CV RMSE	CV RMSE std	CV R ²	Test MAE	Test RMSE	Test R ²
XGBoost	7.40	10.35	0.66	0.922	7.07	9.87	0.930
Gradient Boosting	7.64	10.59	0.70	0.918	7.40	10.39	0.923
Random Forest	8.15	11.48	0.86	0.904	7.95	11.21	0.910
Ridge Regression	12.16	14.76	0.86	0.842	10.01	12.78	0.883
Decision Tree	10.87	15.80	0.62	0.818	9.98	14.67	0.846

Table C.4: Permutation-importance shares of all features

Feature	Academic share	Readiness share	Probability share
CGPA	83.1 %	14.9 %	16.8 %
Attendance %	8.5 %	2.0 %	1.0 %
Study hours / week	4.7 %	0.0 %	0.1 %
Backlogs	1.8 %	6.3 %	3.7 %
Projects	0.7 %	14.1 %	16.8 %
Coding / DSA score	0.7 %	19.6 %	20.1 %

Feature	Academic share	Readiness share	Probability share
Communication score	0.4 %	4.3 %	2.8 %
Aptitude score	0.1 %	7.1 %	5.8 %
Extracurricular score	0.0 %	1.1 %	0.5 %
Certifications	0.0 %	5.5 %	2.9 %
Technical skills score	0.0 %	5.4 %	10.2 %
Programming languages	0.0 %	0.8 %	0.1 %
Internship	0.0 %	18.3 %	17.5 %
Hackathon participation	0.0 %	0.6 %	1.9 %

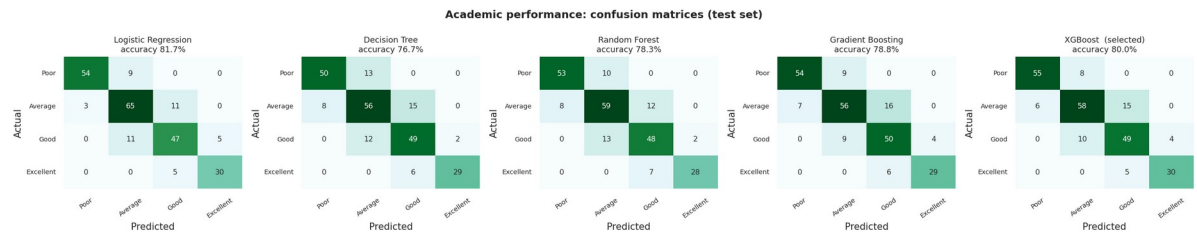


Figure C.1: Confusion matrices of all five academic-performance models (test set)

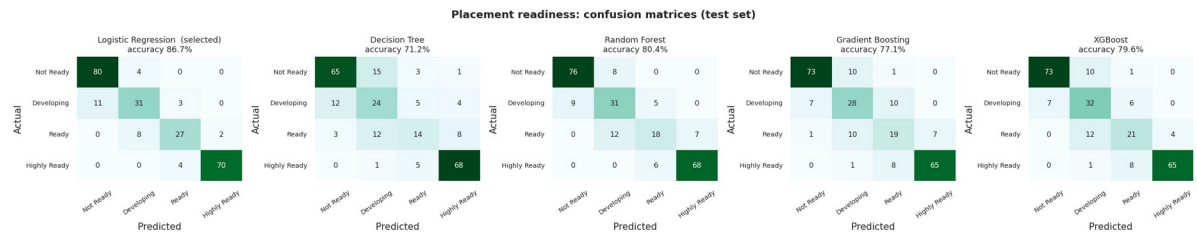
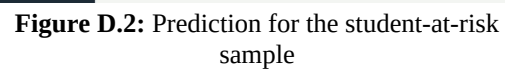


Figure C.2: Confusion matrices of all five placement-readiness models (test set)

The Decision Tree confuses the two middle classes most often.

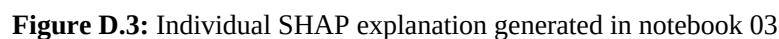


Feature = value

SHAP contribution

Feature = value	SHAP contribution (percentage points)
Technical skills score = 68	9.5
CGPA = 7.4	9.0
Coding / DSA score = 64	6.2
Internship = No	-4.8
Aptitude score = 72	3.5
Communication score = 70	3.2
Backlogs = 0	2.8
Hackathon participation = Yes	2.3

Contribution (percentage points)



55